‘reHỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN
IOT VÀ ỨNG DỤNG

Giảng viên hướng dẫn	**:**	**TS. Nguyễn Quốc Uy**
Nhóm	**:**	**12**
Họ tên và mã sinh viên	**:**	**Nguyễn Minh Đức - B23DCAT050**

Hà Nội – 2026

LỜI NÓI ĐẦU

Trong những năm gần đây, Internet of Things (IoT) được ứng dụng ngày càng rộng rãi trong các hệ thống giám sát và điều khiển từ xa. Việc kết hợp cảm biến, thiết bị điều khiển, giao thức truyền thông và ứng dụng Web giúp người dùng có thể theo dõi trạng thái môi trường và tác động lên thiết bị một cách trực quan.

Trong khuôn khổ bài tập lớn môn IoT và Ứng dụng, báo cáo xây dựng đặc tả cho hệ thống giám sát nhiệt độ, độ ẩm và ánh sáng, đồng thời cho phép người dùng bật hoặc tắt các thiết bị từ giao diện Web. Hệ thống sử dụng ESP32 ở phía phần cứng, Eclipse Mosquitto làm MQTT Broker, Backend để xử lý nghiệp vụ và lưu dữ liệu MySQL, còn Frontend Web dùng để hiển thị và tương tác với người dùng.

Báo cáo tập trung vào các Use Case chính của hệ thống, Sequence Diagram tương ứng, thiết kế cơ sở dữ liệu, giao diện Figma và tài liệu API. Nội dung được trình bày theo hướng mô tả rõ điều kiện, luồng xử lý và sự trao đổi dữ liệu giữa các thành phần, làm cơ sở cho quá trình triển khai hệ thống.

Em xin chân thành cảm ơn TS. Nguyễn Quốc Uy đã hướng dẫn và góp ý trong quá trình thực hiện bài tập lớn. Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót; em mong nhận được thêm ý kiến để tiếp tục hoàn thiện hệ thống.

MỤC LỤC

LỜI NÓI ĐẦU 2
CHƯƠNG 1: TỔNG QUAN 4
1.1. Giới thiệu 4
1.2. Công nghệ sử dụng 4
CHƯƠNG 2: THIẾT KẾ TỔNG QUAN 5
2.1 Kiến trúc tổng quan 5
2.2 Thiết kế phần cứng 6
2.3 Thiết kế phần mềm 6
2.4 Thiết kế cơ sở dữ liệu 7
2.5 Thiết kế luồng dữ liệu, giao tiếp và MQTT topic 9
CHƯƠNG 3: USE CASE 11
3.1. UC1 – Đăng nhập 11
3.2. UC2 – Giám sát phòng học theo thời gian thực 15
3.3. UC3 – Xem dữ liệu cảm biến 18
3.4. UC4 – Bật/Tắt thiết bị 22
3.5 UC5 – Xem lịch sử điều khiển thiết bị 27

CHƯƠNG 1: TỔNG QUAN

1.1. Giới thiệu

Trong xu hướng phát triển của Internet of Things (IoT), việc ứng dụng các thiết bị cảm biến và kết nối mạng vào quản lý cơ sở vật chất giúp nâng cao khả năng giám sát và tự động hóa. Đối với phòng học, các thông tin như nhiệt độ, độ ẩm, ánh sáng và tình trạng có người có thể thay đổi liên tục trong quá trình sử dụng. Việc theo dõi và điều khiển các thiết bị như đèn, quạt theo phương pháp thủ công còn tồn tại nhiều hạn chế.

Đề tài **"Xây dựng hệ thống giám sát và điều khiển phòng học thông minh"** được thực hiện nhằm xây dựng một hệ thống IoT cho phép thu thập dữ liệu từ các cảm biến trong phòng học, truyền dữ liệu về hệ thống trung tâm và hiển thị cho người dùng thông qua giao diện web.

Hệ thống cho phép Admin quản lý phòng học, tài khoản và phân quyền người dùng. Người dùng được Admin cấp quyền có thể truy cập những phòng được cho phép, theo dõi dữ liệu cảm biến và điều khiển các thiết bị như đèn, quạt. Bên cạnh đó, hệ thống hỗ trợ tự động hóa thiết bị dựa trên các điều kiện được thiết lập và phát sinh cảnh báo khi các thông số trong phòng vượt ngưỡng cho phép.

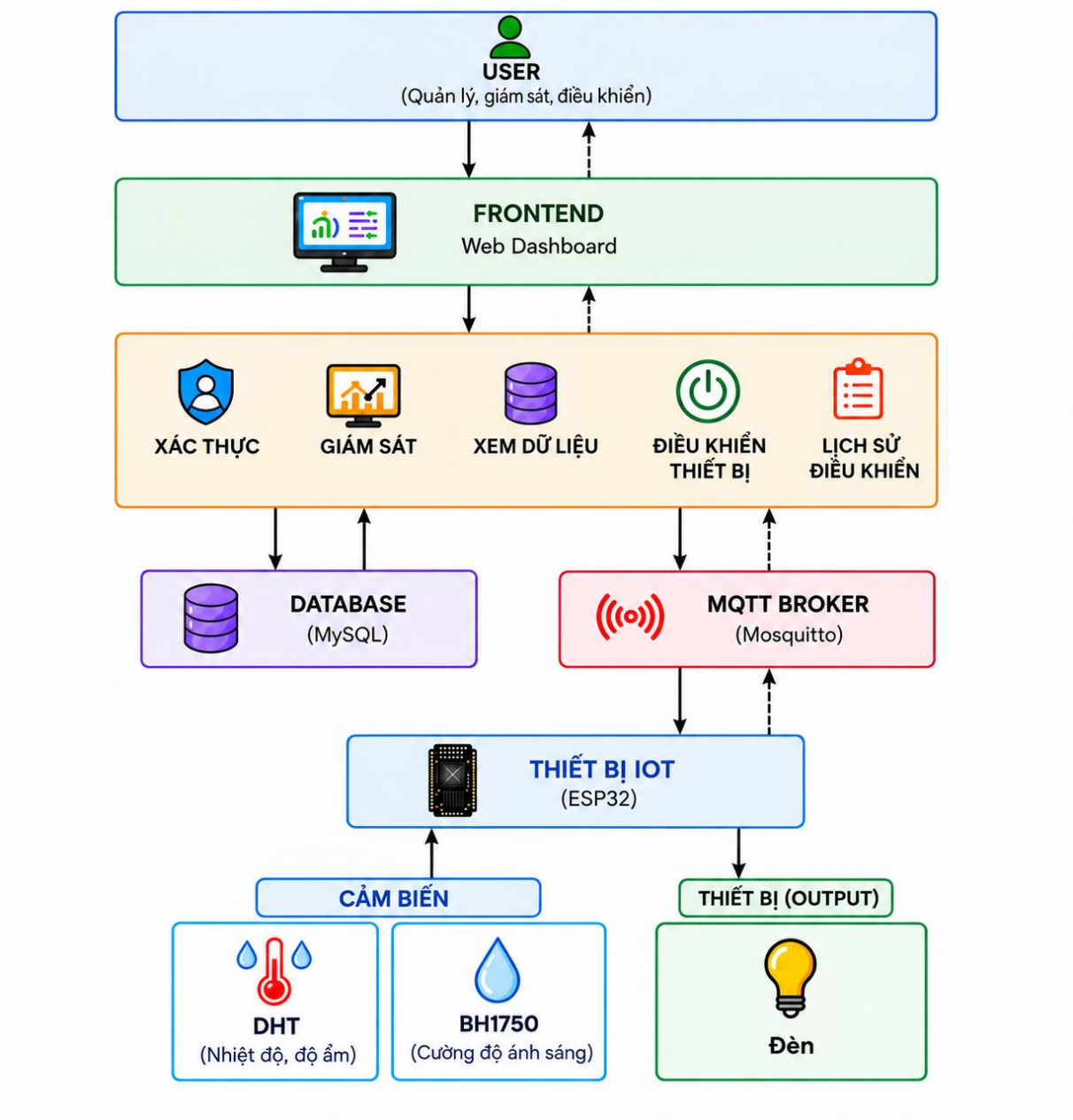
Dữ liệu cảm biến, trạng thái thiết bị, lịch sử điều khiển và cảnh báo được lưu trữ trong cơ

1.2. Công nghệ sử dụng

Thành phần	Công nghệ	Mục đích
Vi điều khiển	ESP32	Thu thập dữ liệu và điều khiển thiết bị
Nhiệt độ/độ ẩm	DHT11	Đo nhiệt độ, độ ẩm
Ánh sáng	BH1750	Đo cường độ ánh sáng
Giao tiếp IoT	MQTT	Trao đổi dữ liệu ESP32 – Backend
MQTT Broker	Mosquitto	Trung gian truyền nhận MQTT
Backend	Django Python	Xây dựng REST API và xử lý nghiệp vụ
Frontend	HTML, CSS, JS	Xây dựng giao diện web
Database	MySQL	Lưu trữ dữ liệu hệ thống
Xác thực	JWT	Xác thực và phân quyền người dùng
Giao tiếp Web	HTTP/HTTPS	Frontend giao tiếp với Backend

CHƯƠNG 2: THIẾT KẾ TỔNG QUAN

2.1 Kiến trúc tổng quan



Hình 2.1: Kiến trúc tổng quan

Các thành phần hệ thống gồm 6 phần:

- Frontend
- Backend
- Database
- MQTT Broker
- ESP32 và cảm biến
- Thiết bị chấp hành

Hệ thống chỉ sử dụng một loại tìa khoản là người dùng. Sau khi đăng nhập có thể thực hiện các chức năng chính của hệ thống

- Đăng nhập vào hệ thống
- Xem dữ liệu cảm biến của phòng học theo thời gian thực
- Xem dữ liệu cảm biến theo thời gian và lựa chọn loại cảm biến
- Bật/tắt các thiết bị trong phòng học
- Xem lịch sử điều khiển thiết bị

2.2 Thiết kế phần cứng

Hệ thống cần đáp ứng

- Đo nhiệt độ
- Đo độ ẩm
- Đo cường độ ánh sáng
- Điều khiển đèn
- Kết nối Wifi
- Gửi dữ liệu lên server

Phòng học sử dụng một ESP32 làm bộ điều khiển trung tâm, thực hiện

- Đọc cảm biến
- Xử lý dữ liệu ban đầu
- Kết nối Wi-fi
- Kết nối MQTT Broker
- Gửi dữ liệu cảm biến
- Nhận lệnh điều khiển
- Cập nhật trạng thái thiết bị

Các thiết bị cảm biến gồm:

- DHT11 cảm biến nhiệt độ, độ ẩm
- BH1750 cảm biến cường độ ánh sáng

Các thiết bị chấp hành gồm:

- Đèn thiết bị chiếu sáng

2.3 Thiết kế phần mềm

a. Thiết kế Frontend
   - Trang đăng nhập
   - Trang tổng quan
   - Trang dữ liệu cảm biến
   - Trang điều khiển thiết bị
   - Trang lịch sử điều khiển

b. Thiết kế Backend
   - Authentication
   - Sensor Monitoring
   - Device Control
   - Device Status

- Control History
- MQTT Communication
- Database Management

2.4 Thiết kế cơ sở dữ liệu

a. Nhóm quản lý tài khoản và phân quyền

Bảng users

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	BIGINT	PK, AUTO_INCREMENT	ID người dùng
username	VARCHAR(50)	UNIQUE, NOT NULL	Tên đăng nhập
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã được hash
full_name	VARCHAR(100)	NOT NULL	Họ và tên
email	VARCHAR(100)	UNIQUE	Email
status	VARCHAR(20)	NOT NULL	Trạng thái tài khoản
created_at	DATETIME	NOT NULL	Thời gian tạo
updated_at	DATETIME	NOT NULL	Thời gian cập nhật
last_login_at	DATETIME	NULL	Thời gian đăng nhập gần nhất

Bảng sensor_readings

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	ID bản ghi
temperature	DECIMAL(5,2)	NOT NULL, CHECK	Nhiệt độ
humidity	DECIMAL(5,2)	NOT NULL, CHECK	Độ ẩm
light_level	DECIMAL(10,2)	NOT NULL, CHECK	Cường độ ánh sáng

Bảng sensor_data

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	ID dữ liệu cảm biến
sensor_type	VARCHAR(50)	NOT NULL, CHECK	Loại cảm biến
value	DECIMAL(10,2)	NOT NULL	Giá trị đo được
recorded_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian ghi nhận

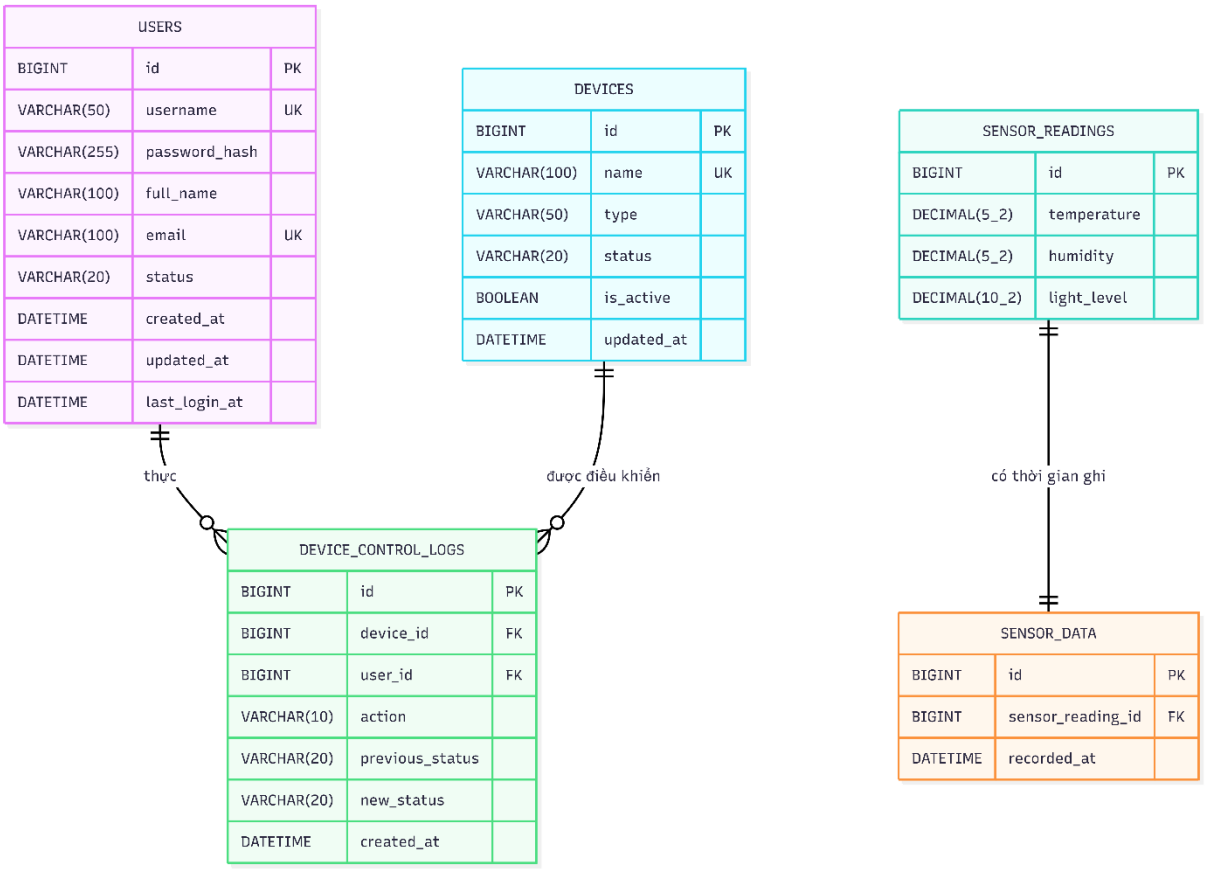
Bảng devices

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	ID thiết bị
name	VARCHAR(100)	NOT NULL, UNIQUE	Tên thiết bị
type	VARCHAR(50)	NOT NULL, CHECK	Loại thiết bị
status	VARCHAR(20)	NOT NULL, DEFAULT 'OFF', CHECK	Trạng thái
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Trạng thái hoạt động
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian cập nhật

Bảng device_control_logs

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	ID lịch sử
device_id	BIGINT	NOT NULL, FK	Thiết bị được điều khiển
user_id	BIGINT	NOT NULL, FK	Người thực hiện
action	VARCHAR(10)	NOT NULL, CHECK	Thao tác ON/OFF
previous_status	VARCHAR(20)	NOT NULL, CHECK	Trạng thái trước
new_status	VARCHAR(20)	NOT NULL, CHECK	Trạng thái sau

created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian thao tác
Cột	Kiểu dữ liệu	Ràng buộc	Mô tả



Hình 2.2: Sơ đồ ERD

Quan hệ giữa các bảng

- users – device_control_logs: quan hệ 1:N.
- devices – device_control_logs: quan hệ 1:N
- sensor_readings: không có khóa ngoại vi

2.5 Thiết kế luồng dữ liệu, giao tiếp và MQTT topic

a. Luồng dữ liệu và giao tiếp

Dữ liệu đi qua hệ thống: Cảm biến -> ESP32 -> MQTT Broker -> Backend -> Database -> Frontend -> User

Đối với chiều điều khiển: User -> Frontend -> Backend -> MQTT Broker -> ESP32 -> Relay -> Đèn

b. Giao tiếp giữa các thành phần

Thành phần	Giao tiếp	Mục đích

ESP32 – MQTT Broker	MQTT	Gửi dữ liệu cảm biến và nhận lệnh
Backend – MQTT Broker	MQTT	Nhận dữ liệu và gửi lệnh điều khiển
Frontend – Backend	HTTP/HTTPS REST API	Giám sát và điều khiển
Backend – Database	SQL	Lưu và truy vấn dữ liệu
MQTT Broker – ESP32	MQTT	Truyền dữ liệu hai chiều

c. MQTT topic sử dụng

Topic	Chiều	Nội dung
iot/sensors/data	ESP32 → Backend	Dữ liệu cảm biến
iot/rooms/{room_id}/devices/control	Backend → ESP32	Lệnh bật/tắt thiết bị
iot/rooms/{room_id}/devices/status	ESP32 → Backend	Trạng thái thiết bị

CHƯƠNG 3: USE CASE

3.1. UC1 – Đăng nhập

a. Đặc tả use case

Thuộc tính	Nội dung
Mã	UC1
Tên	Đăng nhập
Actor	Người dùng
Mục đích	Cho phép người dùng xác thực tài khoản để truy cập vào hệ thống giám sát và điều khiển phòng học.
Trigger	Người dùng truy cập hệ thống và thực hiện chức năng đăng nhập
Tiền điều kiện	• Người dùng đã có tài khoản • Backend đang hoạt động • Database có thể truy cập được
Hậu điều kiện	• Đăng nhập thành công: người dùng được xác thực và được phép truy cập các chức năng của hệ thống • Đăng nhập thất bại: người dùng vẫn ở màn hình đăng nhập và nhận thông báo

Luồng sự kiện chính

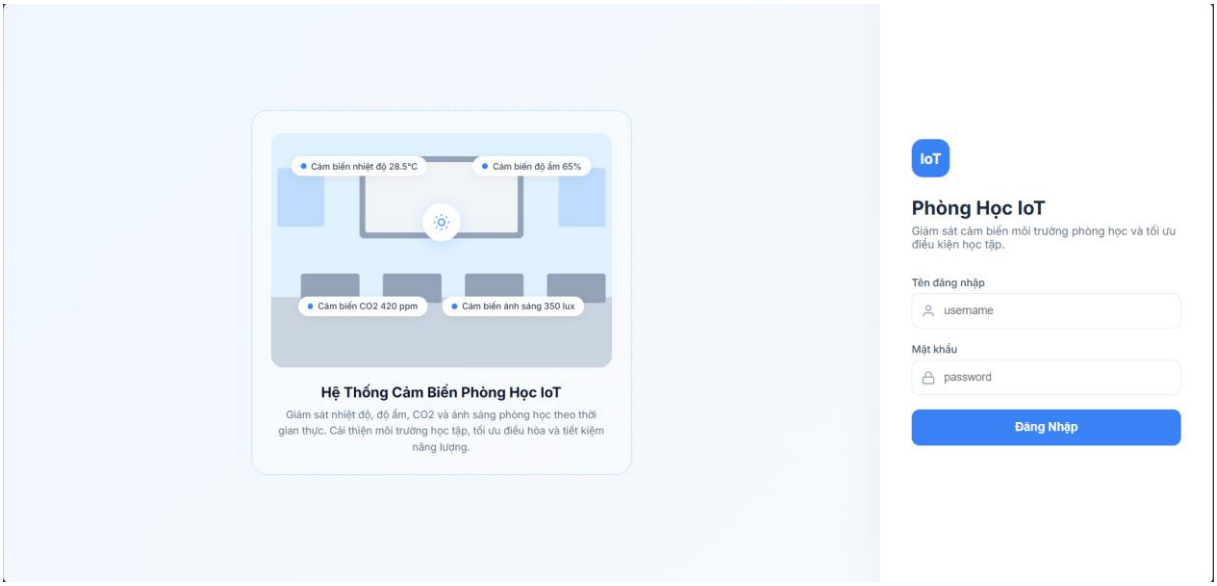
1. Người dùng truy cập trang đăng nhập
2. Hệ thống hiển thị biểu mẫu đăng nhập gồm tên đăng nhập và mật khẩu
3. Người dùng nhập tên đăng nhập và mật khẩu
4. Người dùng nhấn nút đăng nhập
5. Frontend gửi thông tin đăng nhập đến backend thông qua API
6. Backend kiểm tra dữ liệu đầu vào
7. Backend tìm tài khoản tương ứng trong bảng users
8. Backend kiểm tra trạng thái tài khoản
9. Backend thực hiện đối chiếu mật khẩu với password_hash được lưu trong database
10. Nếu thông tin hợp lệ, backend tạo access token
11. Backend cập nhật thời gian đăng nhập gần nhất của người dùng
12. Backend trả kết quả đăng nhập thành công cho frontend

13. Frontend lưu access token và chuyển người dùng đến giao diện giám sát phòng h

Luồng ngoại lệ

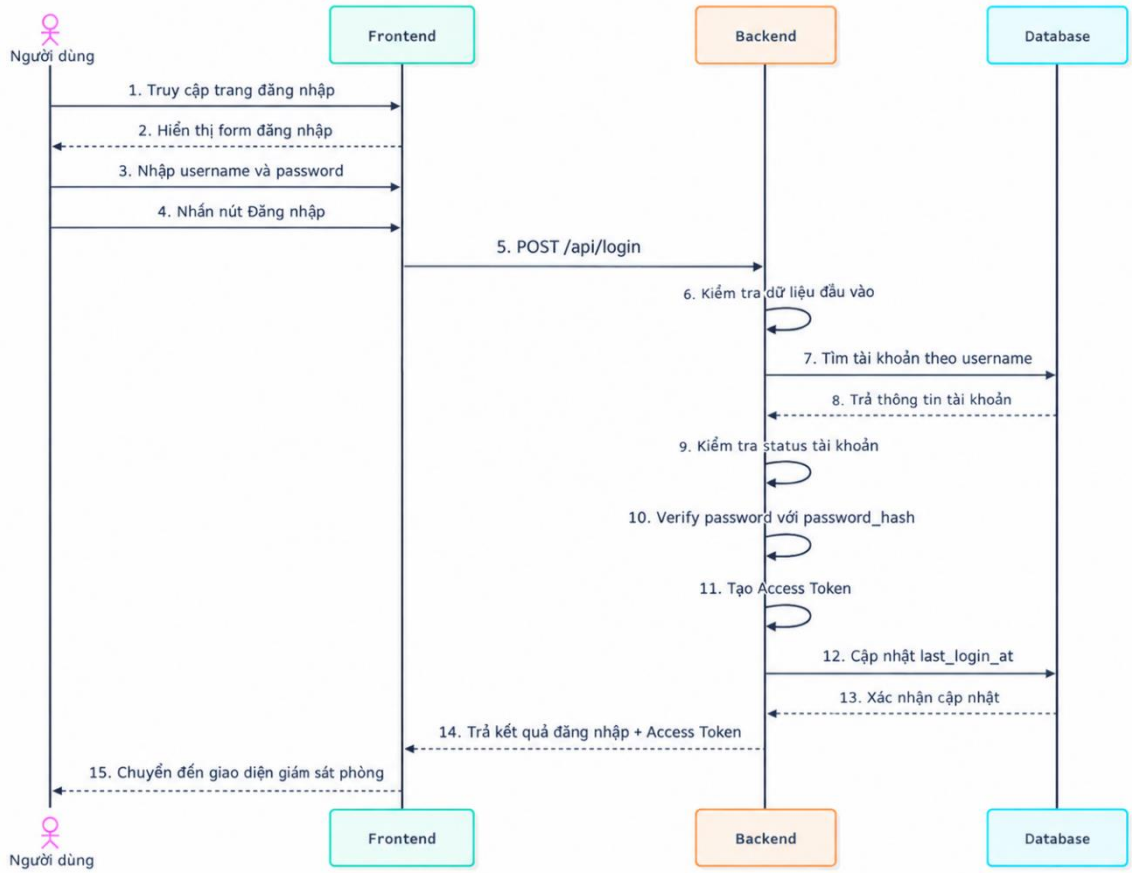
1. Sai tên đăng nhập hoặc sai mật khẩu: backend trả thông báo tài khoản không tồn tại hoặc thông tin đăng nhập không chính xác
2. Thiếu thông tin: nếu người dùng không nhập tên đăng nhập hoặc mật khẩu frontend sẽ yêu cầu nhập đầy đủ thông tin
3. Lỗi hệ thống: nếu backend hoặc database lỗi hệ thống sẽ thông báo không thể thực hiện đăng nhập và người dùng có thể thử lại

b. Các giao diện liên quan



Hình 3.1: Giao diện đăng nhập

c. Sequence Diagram



Hình 3.2: Sequence Đăng Nhập

Mô tả Sequence Diagram – UC1: Đăng nhập

1. Người dùng truy cập vào hệ thống và lựa chọn chức năng đăng nhập
2. Frontend hiển thị giao diện đăng nhập gồm 2 trường username và password
3. Người dùng nhập tên đăng nhập và mật khẩu vào biểu mẫu
4. Người dùng nhấn nút đăng nhập
5. Frontend gửi yêu cầu POST /api/auth/login đến backend, trong request body chứa username và password
6. Backend kiểm tra dữ liệu đầu vào, đảm bảo username và password được cung cấp đầy đủ và đúng dudnhj dạng
7. Backend truy vấn bảng users để tìm tài khoản có username tương ứng
8. Database trả về thông tin tài khoản bao gồm id, username, password_hash, full_name, status
9. Backend kiểm tra trạng thái tài khoản thông qua cột status. Nếu tài khoản không hoạt động, quá trình đăng nhập bị từ chối
10. Backend thực hiện xác thực mật khẩu bằng cách đối chiếu mật khẩu người dùng với passwortd_hash được lưu trong database
11. Nếu mật khẩu chính xác, backend tạo access token để sử dụng cho các request tiếp theo
12. Backend cập nhật cột last_login_at trong bảng users để lưu thời điểm đăng nhập gần nhất
13. Database thực hiện cập nhật và trả kết quả xác nhận cho backend

14. Backend trả response đăng nhập thành công cho frontend trong đó có thông tin người dùng và access token
15. Frontend nhận kết quả lưu access token và chuyển người dùng đến giao diện giám sát và điều khiển phòng học

d. Xây dựng API

API chính của Use case này sẽ là POST /api/ login

Thuộc tính	Nội dung
Header	Content-Type: application/json

Body	{
	"username": "admin",
	"password": "admin"
	}

Status	HTTP/1.1 200 OK

Request	{
	"success": true,
	"message": "Đăng nhập thành công",
	"data": {
	"access_token": "eyJhbGciOiJIUzI1NiIs...",
	"user": {
	"id": 1,
	"username": "admin",
	"full_name": "System Administrator",
	}
	}
	}

e. Mapping API đến database

API	Dữ liệu request	Bảng database	Cột database	Mục đích
POST /api/login	username	users	username	Tìm tài khoản
POST /api/login	password	users	password_hash	Đối chiếu mật khẩu sau khi hash/verify
POST /api/login		users	status	Kiểm tra tài khoản có hoạt động

POST /api/login		users	id	Xác định người dùng
POST /api/login		users	full_name	Trả thông tin người dùng
POST /api/login		users	Last_login_at	Cập nhật thời gian đăng nhập

3.2. UC2 – Giám sát phòng học theo thời gian thực

a. Đặc tả use case

Thuộc tính	Nội dung
Mã	UC2
Tên	Giám sát phòng học theo thời gian trực
Actor	Người dùng
Mục đích	Cho phép người dùng theo dõi các thông số môi trường hiện tại của phòng học theo thời gian thực
Trigger	Người dùng truy cập dashboard
Tiền điều kiện	• Người dùng đã đăng nhập thành công • Access Token còn hiệu lực • Backend còn hoạt động • ESP32, cảm biến và MQTT Broker đang hoạt động
Hậu điều kiện	Các giá trị nhiệt độ, độ ẩm, ánh sáng và thời gian cập nhật mới nhất được hiển thị trên giao diện

Luồng sự kiện chính

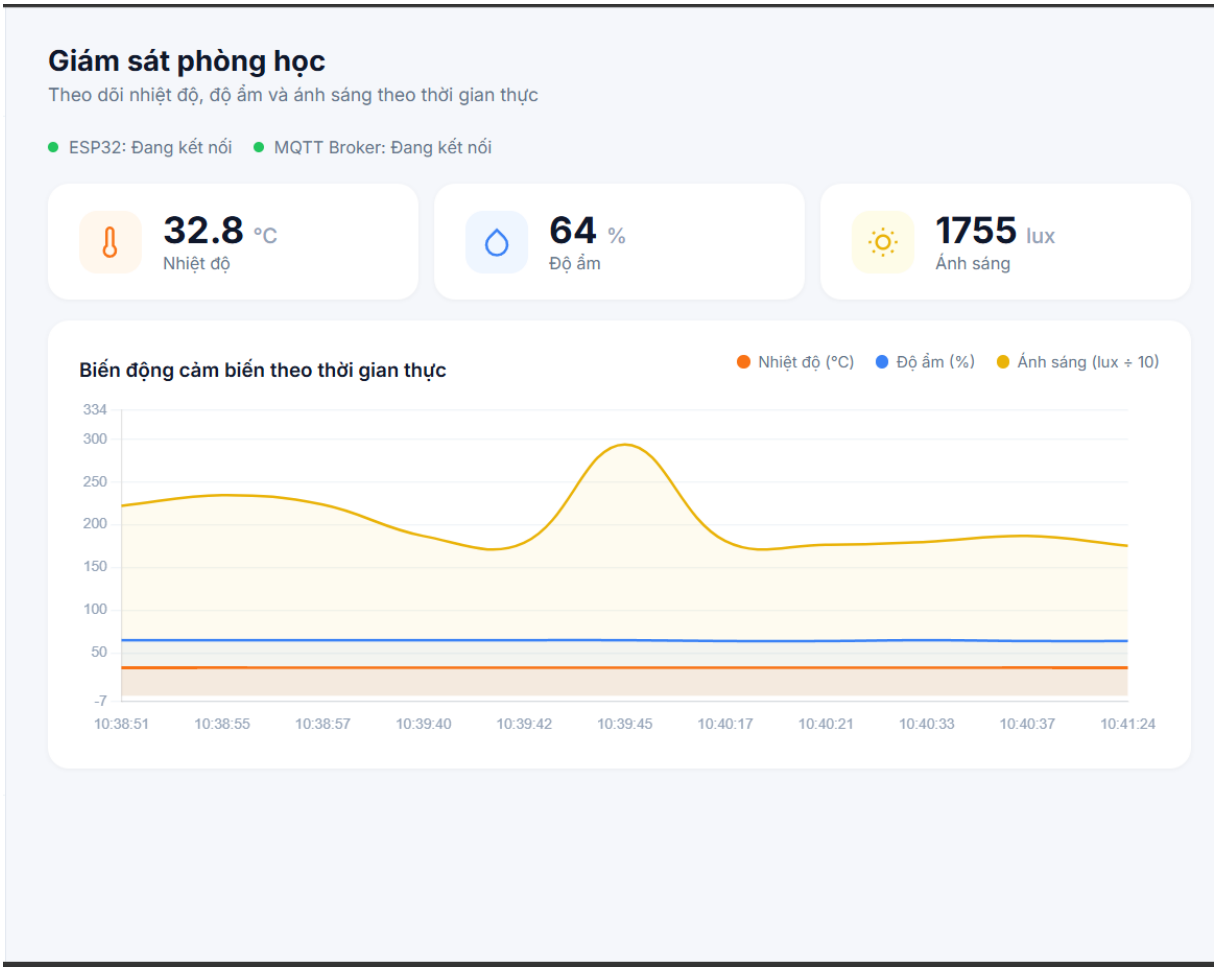
1. Người dùng truy cập giao diện giám sát phòng học

2. Frontent gửi yêu cầu lấy dữ liệu cảm biến mới nhất trên backend
3. Backend xác thực access token
4. Backend truy vấn database để lấy dữ liệu cảm biến mới nhất
5. Database trả về nhiệt độ, độ ẩm, ánh sáng và thời gian ghi nhận
6. Backend trả dữ liệu về frontend hiển thị thông số hiện tại của người dùng
7. ESP32 đọc dữ liệu từ DHT11 và BH1750
8. ESP32 nhận dữ liệu nhiệt độ, độ ẩm và ánh sáng gửi dữ liệu cùng thời gian ghi nhận lên MQTT Broker và chuyển đến backend
9. Backend kiểm tra và xử lý dữ liệu sau đó lưu vào bảng sensor_readings
10. Backend gửi dữ liệu mới đến frontend để cập nhật dữ liệu trên giao diện

Luồng ngoại lệ

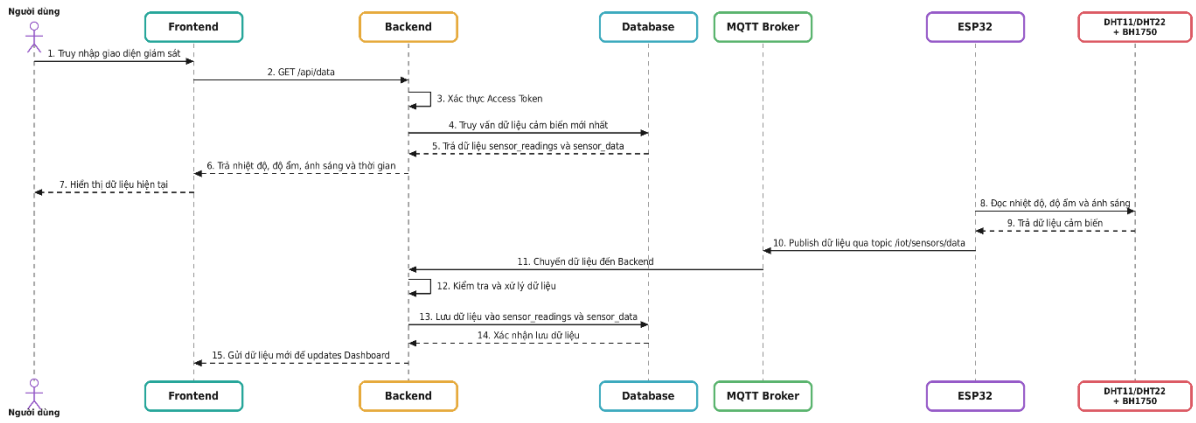
1. ESP32 mất kết nối khiến hệ thống không nhận được dữ liệu mới
2. MQTT Broker mất kết nối, dữ liệu không được chuyển đến backend
3. Cảm biến không trả về dữ liệu hợp lệ, backend không lưu bản ghi

b. Các giao diện liên quan



Hình 3.3: Giao diện giám sát phòng học theo thời gian thực

c. Sequence Diagram



Hình 3.4: Sequence giám sát phòng học theo thời gian thực

Mô tả Sequence Diagram – UC2: giám sát phòng học theo thời gian thực

1. Người dùng truy cập vào giao diện dashboard
2. Frontend gửi request GET /api/v1/sensors/latest đến backend để lấy dữ liệu cảm biến mới nhất
3. Backend kiểm tra và xác thực access token của người dùng
4. Backend truy vấn bảng sensor_readings để lấy bản ghi cảm biến mới nhất
5. Database trả về các dữ liệu liên quan từ bảng sensor_readings và sensor_data, bao gồm các giá trị nhiệt độ, độ ẩm, ánh sáng và thời gian ghi nhận.
6. Backend trả dữ liệu cảm biến mới nhất về frontend
7. Frontend hiển thị các thông số hiện tại cho người dùng
8. ESP32 thực hiện đọc dữ liệu cảm biến từ DHT11 và BH1750
9. Các cảm biến trả về giá trị nhiệt độ, độ ẩm và cường độ ánh sáng cho ESP32
10. ESP32 đóng gói dữ liệu cảm biến cùng thời gian ghi nhận và publish lên MQTT Broker thông qua topic iot/sensors/data
11. MQTT Broker nhận dữ liệu và chuyển message đến backend
12. Backend tiếp nhận dữ liệu, kiểm tra tính hợp lệ và xử lý dữ liệu cảm biến
13. Backend lưu dữ liệu vào database, trong đó các giá trị cảm biến được lưu vào bảng sensor_readings và thông tin thời gian ghi nhận được lưu vào bảng sensor_data
14. Database xác nhận dữ liệu đã được lưu thành công
15. Backend gửi dữ liệu mới đến frontend cập nhật lại thông tin trên giao diện

d. Xây dựng API

Thuộc tính	Nội dung
Request	GET /api/dashboard Authorization: Bearer <access_token>
Status	200 OK

Response	{
	"data": {
	"temperature": 28.5,
	"humidity": 65.2,

	"light_level": 350, "recorded_at": "2026-08-29T10:30:15Z" } }

e. Mapping API đến database

API	Dữ liệu request	Bảng Database	Cột Database	Mục đích
GET /api/dashboard		sensor_readings	temperature	Lấy nhiệt độ mới nhất
GET /api/dashboard		sensor_readings	humidity	Lấy độ ẩm mới nhất
GET /api/dashboard		sensor_readings	light_level	Lấy cường độ ánh sáng mới nhất
GET /api/dashboard		sensor_data	recorded_at	Lấy thời gian ghi nhận
MQTT iot/sensor/data	temperature	sensor_readings	temperature	Lưu nhiệt độ
MQTT iot/sensor/data	humidity	sensor_readings	humidity	Lưu độ ẩm
MQTT iot/sensor/data	light	sensor_readings	light_level	Lưu ánh sáng
MQTT iot/sensor/data	recorded_at	sensor_data	recorded_at	Lưu thời gian ghi nhận

3.3. UC3 – Xem dữ liệu cảm biến

a. Đặc tả use case

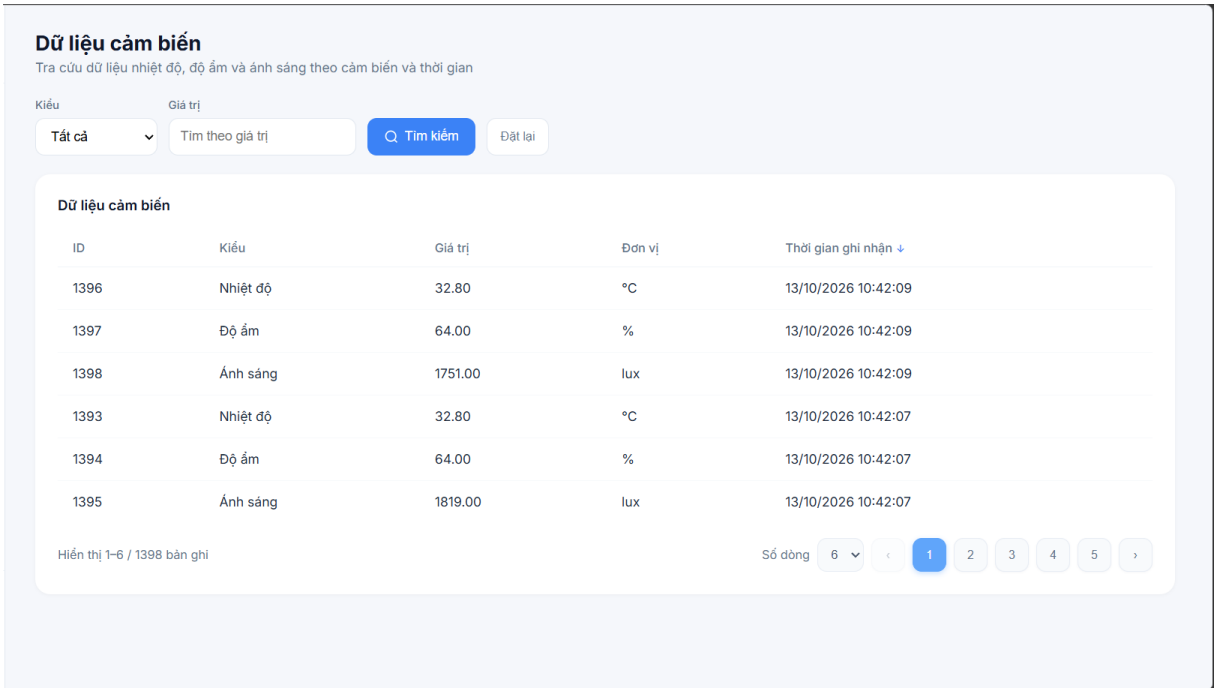
Thuộc tính	Nội dung
Mã	UC3
Tên	Xem dữ liệu cảm biến
Actor	Người dùng

Thuộc tính	Nội dung
Mục đích	Cho phép người dùng xem và tra dữ liệu nhiệt độ, độ ẩm và ánh sáng được lưu trong hệ thống
Trigger	Người dùng truy cập chức năng xem dữ liệu cảm biến
Tiền điều kiện	• Người dùng đã đăng nhập thành công • Access Token còn hiệu lực • Backend đang hoạt động • Database có dữ liệu cảm biến
Hậu điều kiện	Dữ liệu cảm biến theo khoảng thời gian được yêu cầu được hiển thị cho người dùng dưới dạng bảng hoặc biểu đồ

Luồng sự kiện chính

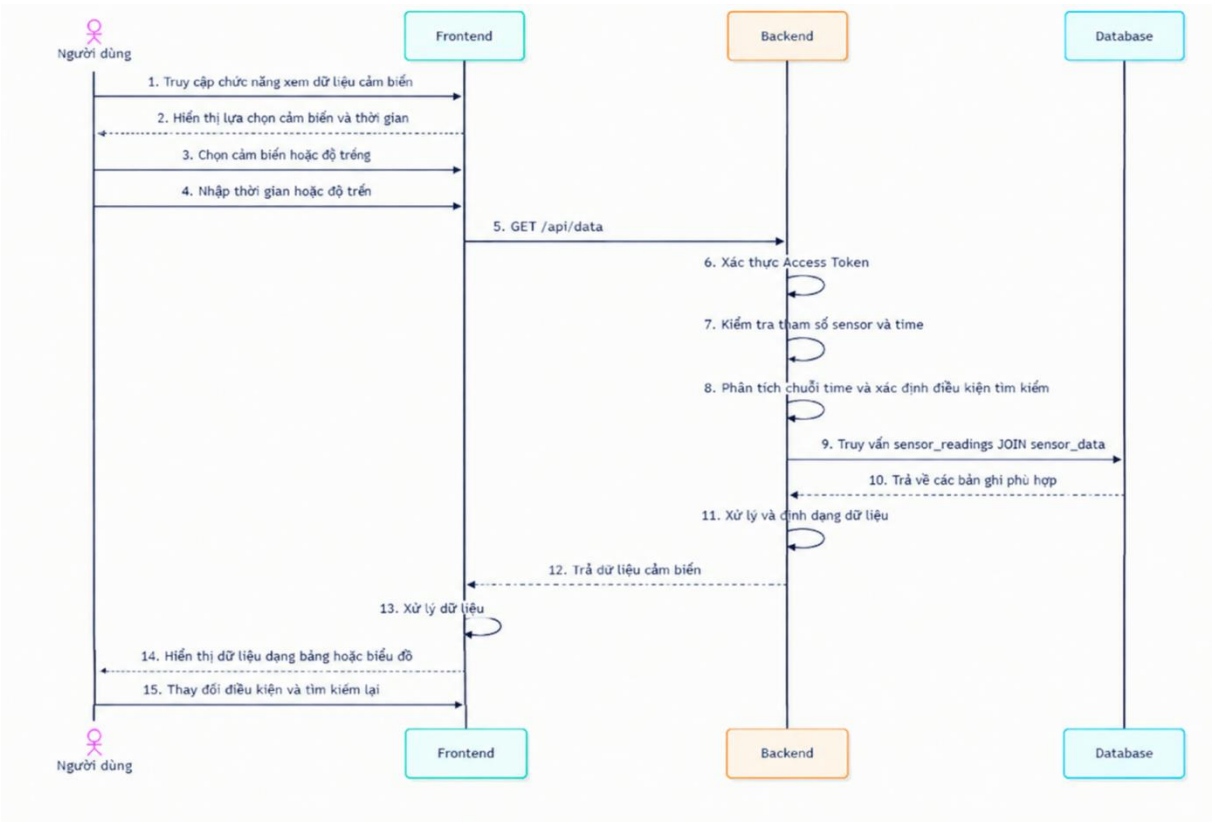
1. Người dùng truy cập chức năng xem dữ liệu cảm biến
2. Hệ thống hiển thị giao diện bao gồm danh sách cảm biến DHT11 và BH1750 cùng ô nhập thời gian tìm kiếm
3. Người dùng có thể chọn cảm biến cần xem hoặc để trống để xem tất cả cảm biến
4. Người dùng có thể nhập thời gian tìm kiếm dưới dạng chuỗi để xem dữ liệu cần tìm
5. Frontend gửi yêu cầu /api/v1/sensors đến backend kèm các tham số sensor và time nếu người dùng nhập
6. Backend kiểm tra giá trị sensor và định dạng chuỗi time nếu được cung cấp
7. Backend xác định phạm vi dữ liệu cần truy vấn
8. Backend thực hiện truy vấn bảng sensor_readings
9. Database trả về các bản ghi cảm biến phù hợp với điều kiện tìm kiếm
10. Backend xử lý và định dạng dữ liệu cảm biến
11. Backend trả dữ liệu về frontend hiển thị dữ liệu dưới dạng biểu đồ

b. Các giao diện liên quan



Hình 3.5: Giao diện dữ liệu cảm biến

c. Sequence Diagram



Hình 3.6: Sequence xem dữ liệu cảm biến

Mô tả Sequence Diagram – UC3: xem dữ liệu cảm biến

1. Người dùng truy cập chức năng xem dữ liệu cảm biến
2. Frontend hiển thị danh sách cảm biến gồm DHT11 và BH1750, đồng thời hiển thị ô nhập thời gian
3. Người dùng có thể chọn một cảm biến hoặc không chọn

4. Người dùng có thể nhập thời gian hoặc để trống
5. Frontend gửi request /api/v1/sensors đến backend với các tham số sensor và time
6. Backend xác thực access token của người dùng
7. Backend kiểm tra các tham số sensor và time
8. Backend phân tích chuỗi time để xác định phạm vi thời gian cần tìm. C huỗi có thể chỉ chứa giờ, giờ và phút, hoặc ngày, giờ và phút
9. Backend xây dựn câu truy vấn và truy vấn bảng sensor_readings, kết hợp với bảng sensor_data để lấy thời gian ghi nhận
10. Database trả về các bản ghi phù hợp với điều kiện tìm kiếm
11. Backend xử lý và định dạng dữ liệu nhận được từ database
12. Backend trả kết quả về frontend
13. Frontend xử lý dữ liệu để chuẩn bị hiển thị
14. Frontend hiển thị dữ liệu cảm biến dưới dạng biểu đồ
15. Người dùng có thể thay đổi cảm biến hoặc thời gian để thực hiện tìm kiếm lại

d. Xây dựng API

Thuộc tính	Nội dung
Method + URL	GET /api/data?sensor=temp&value=28&page=1&page_size=6&sort=time&order=desc
Header	Authorization: Bearer <access_token>
Response	{ "data": [{ "id": 1, "sensor_type": "temperature", "value": "28.50", "recorded_at": "2026-10-13T10:30:15+07:00" }], "count": 36, "page": 1, "page_size": 6 }

e. Mapping API đến database

Querry:

Tham số	Ví dụ	Mô tả
sensor	all | temp | hum | light | time	Lọc theo kiểu cảm biến
value	28 hoặc 10:30	Chuỗi tìm trong giá trị hoặc thời gian

sort	time | id	Trường sắp xếp (mặc định time)
order	asc | desc	Chiều sắp xếp (mặc định desc)
page	1	Số trang
page_size	4 | 6 | 8	Số dòng mỗi trang

API	Request	Bảng	Cột	Mục đích
GET /api/data	sensor	sensor_data	sensor_type	Lọc temperature / humidity / light
GET /api/data	value	sensor_data	value	Tìm theo giá trị (chứa)
GET /api/data	value (sensor=time)	sensor_data	recorded_at	Tìm theo chuỗi thời gian
GET /api/data	sort, order	sensor_data	id / recorded_at	Sắp xếp
GET /api/data	page, page_size			Phân trang

3.4. UC4 – Bật/Tắt thiết bị

a. Đặc tả use case

Thuộc tính	Nội dung
Mã	UC4
Tên	Xem lịch sử và thống kê
Actor	Người dùng
Mục tiêu	Cho phép người dùng điều khiển trạng thái bật/tắt của các thiết bị trong phòng học
Trigger	Người dùng chọn chức năng điều khiển thiết bị và thực hiện bật hoặc tắt thiết bị
Tiền điều kiện	• Người dùng đã đăng nhập thành công vào hệ thống • Access Token còn hiệu lực • Backend đang hoạt động • Thiết bị IoT và MQTT Broker đang hoạt động • Thiết bị cần điều khiển đã được đăng ký trong hệ thống

Thuộc tính	Nội dung
Mã	UC4
Hậu điều kiện	• Trạng thái thiết bị được cập nhật thành công • Thiết bị thực tế thay đổi trạng thái tương ứng • Thông tin thao tác được lưu vào database

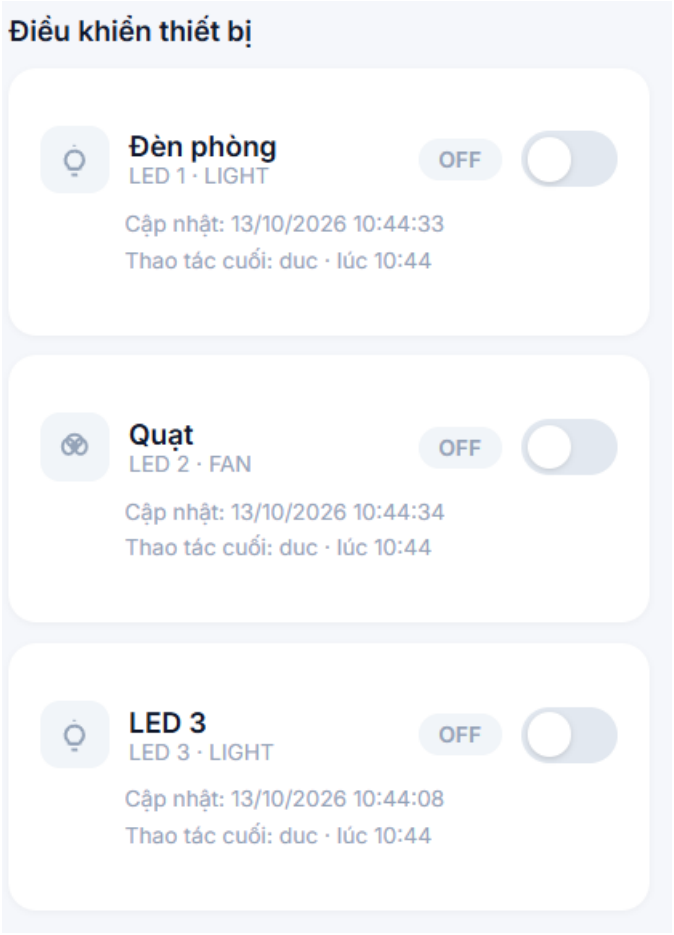
Luồng sự kiện chính

1. Người dùng truy cập chức năng bật/tắt thiết bị
2. Frontend hiển thị danh sách thiết bị trong phòng cùng trạng thái hiện tại
3. Người dùng chọn thiết bị cần điều khiển
4. Frontend gửi yêu cầu điều khiển thiết bị đến backend
5. Backend xác thực access token người dùng
6. Backend kiểm tra thiết bị đó có tồn tịa không và kiểm tra thao tác yêu cầu là ON hay OFF
7. Backend gửi lệnh điều khiển đến MQTT Broker
8. MQTT Broker chuyển lệnh đến ESP32 nhận và thực hiển lệnh điều khiển thiết bị
9. ESP32 cập nhật trạng thái thực tế của thiết bị và phản hồi về MQTT Broker
10. Backend nhận trạng thái mới và cập nhật bảng devices
11. Backend lưu thông tin thao tác vào device_control_logs
12. Backend trả kết quả về frontend và hiển thị trạng thái mới cho người dùng

Luồng ngoại lệ

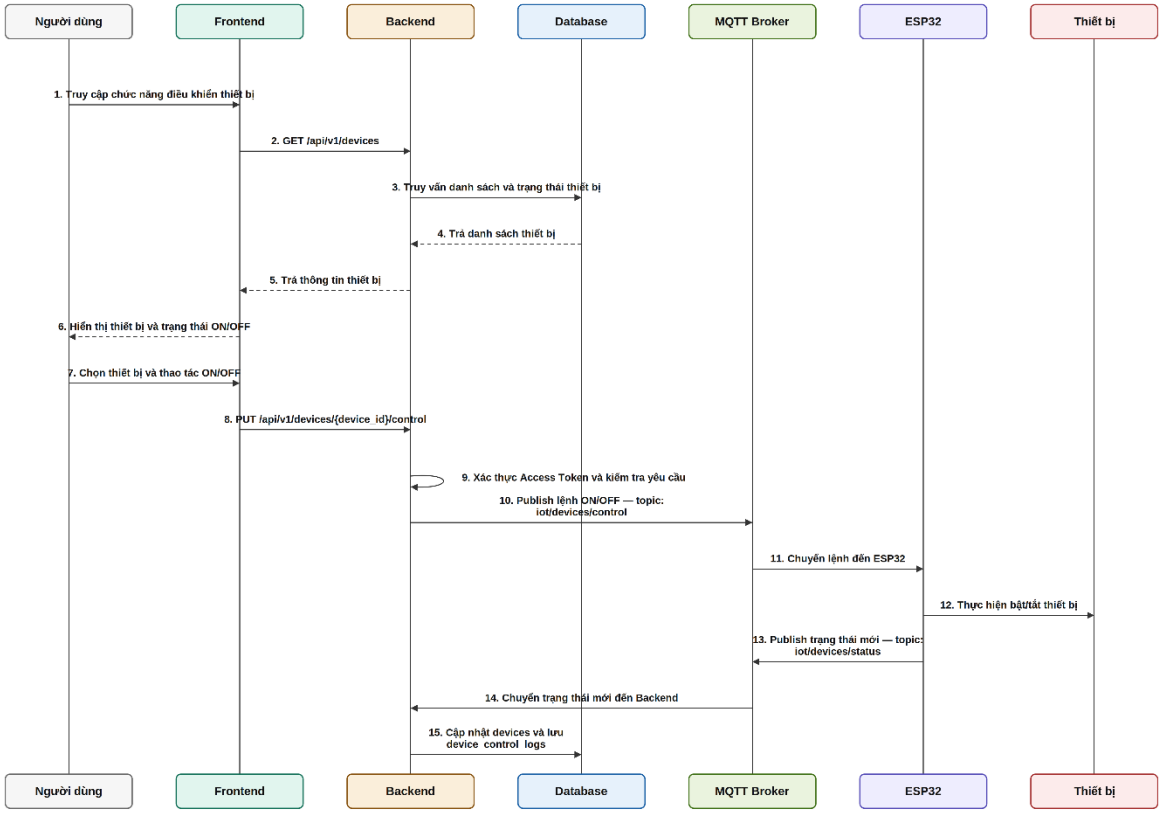
1. Thiết bị không tồn tại hoặc không hoạt động, backend trả lỗi và không gửi lệnh
2. Thao tác không hợp lệ backend trả 400 Bad Request
3. MQTT Broker không hoạt động không gửi lệnh đến ESP32
4. ESP32 không phản hồi thông báo điều khiển thiết bị thất bại

b. Các giao diện liên quan



Hình 3.7: Giao diện bật/tắt thiết bị

c. Sequence Diagram



Hình 3.8: Sequence bật/tắt thiết bị

Mô tả Sequence Diagram – UC4: bật/tắt thiết bị

1. Người dùng chọn chức năng điều khiển thiết bị
2. Frontend gửi yêu cầu /api/v1/devices đến backend lấy danh sách thiết bị
3. Backend truy vấn database để lấy thông tin các thiết bị đang hoạt động
4. Database trả về danh sách thiết bị cùng trạng thái hiện tại cho backend
5. Backend trả danh sách thiết bị và trạng thái hiện tại về frontend
6. Frontend hiển thị danh sách phòng học cùng các trạng thái ON/OFF
7. Người dùng chọn thiết bị cần điều khiển và thực hiện thao tác bật hoặc tắt
8. Frontend gửi yêu cầu PUT /api/v1/devices/{device_id}/control đến backend kèm theo thao tác ON hoặc OFF
9. Backend truy vấn database để lấy trạng thái hiện tại của thiết bị
10. Database trả trạng thái hiện tại
11. Backend gửi ON/OFF đến MQTT Broker
12. MQTT Broker chuyển lệnh đến ESP32
13. ESP32 thực hiện lệnh bật/tắt
14. Thiết bị trả trạng thái mới về ESP32
15. ESP32 Publish trạng thái mới lên MQTT Broker
16. MQTT Broker chuyển trạng thái mới về backend
17. Backend cập nhật trạng thái mới về bảng devices
18. Backend lưu thông tin thao tác vào device_control_logs, bao gồm người dùng, thiết bị, thao tác, trạng thái trước, sau và thời gian thực hiện
19. Database xác nhận việc cập nhật trạng thái và lưu lịch sử thành công
20. Backend trả kết quả về frontend, sau đó frontend hiển thị trạng thái mới cho người dùng

d. Xây

Thuộc tính	Nội dung
Method + URL	GET /api/devices
Body	{action:”OFF”}
Header	Authorization: Bearer <access_token>

Response	{
	"data": [
	{
	"id": 1,
	"name": "Đèn phòng",
	"type": "LIGHT",
	"status": "OFF",
	"is_active": true,
	"updated_at": "2026-10-13T10:31:00+07:00"
	}
	]
	}

e.

f. Mapping API đến database

API	Dữ liệu request	Bảng Database	Cột Database	Mục đích
GET /api/v1/devices		devices	id	Xác định thiết bị
GET /api/v1/devices		devices	name	Lấy tên thiết bị
GET /api/v1/devices		devices	type	Xác định loại thiết bị
GET /api/v1/devices		devices	status	Lấy trạng thái hiện tại
GET /api/v1/devices		devices	is_active	Kiểm tra thiết bị đang hoạt động
GET /api/v1/devices		devices	updated_at	Xác định thời gian cập nhật trạng thái

API	Dữ liệu request	Bảng Database	Cột Database	Mục đích
PUT /api/v1/devices/{device_id}/control	device_id	devices	id	Xác định thiết bị cần điều khiển
PUT /api/v1/devices/{device_id}/control	action	devices	status	Cập nhật trạng thái thiết bị
PUT /api/v1/devices/{device_id}/control		devices	is_active	Kiểm tra thiết bị có hoạt động
PUT /api/v1/devices/{device_id}/control		devices	updated_at	Lưu thời gian cập nhật
PUT /api/v1/devices/{device_id}/control		device_control_logs	device_id	Xác định thiết bị được thao tác

PUT /api/v1/devices/{device_id}/control		device_control_logs	user_id	Xác định người thực hiện thao tác
PUT /api/v1/devices/{device_id}/control	action	device_control_logs	action	Lưu thao tác ON/OFF
PUT /api/v1/devices/{device_id}/control		device_control_logs	previous_status	Lưu trạng thái trước khi thao tác
PUT /api/v1/devices/{device_id}/control		device_control_logs	new_status	Lưu trạng thái sau khi thao tác
PUT /api/v1/devices/{device_id}/control		device_control_logs	created_at	Lưu thời gian thao tác

3.5 UC5 – Xem lịch sử điều khiển thiết bị

a. Đặc tả use case

Thuộc tính	Nội dung
Mã	UC5
Tên	Xem lịch sử điều khiển thiết bị
Actor	Người dùng
Mục tiêu	Cho phép người dùng xem lại lịch sử các thao tác bật/tắt thiết bị trong phòng họ
Trigger	Người dùng truy cập chức năng lịch sử điều khiển thiết bị

Thuộc tính	Nội dung
Mã	UC5
Tiền điều kiện	• Người dùng đã đăng nhập thành công vào hệ thống • Access Token còn hiệu lực • Backend đang hoạt động • Database có thể truy cập dược
Hậu điều kiện	• Lịch sử điều khiển thiết bị được hiển thị cho người dùng • Không làm thay đổi dữ liệu trong database

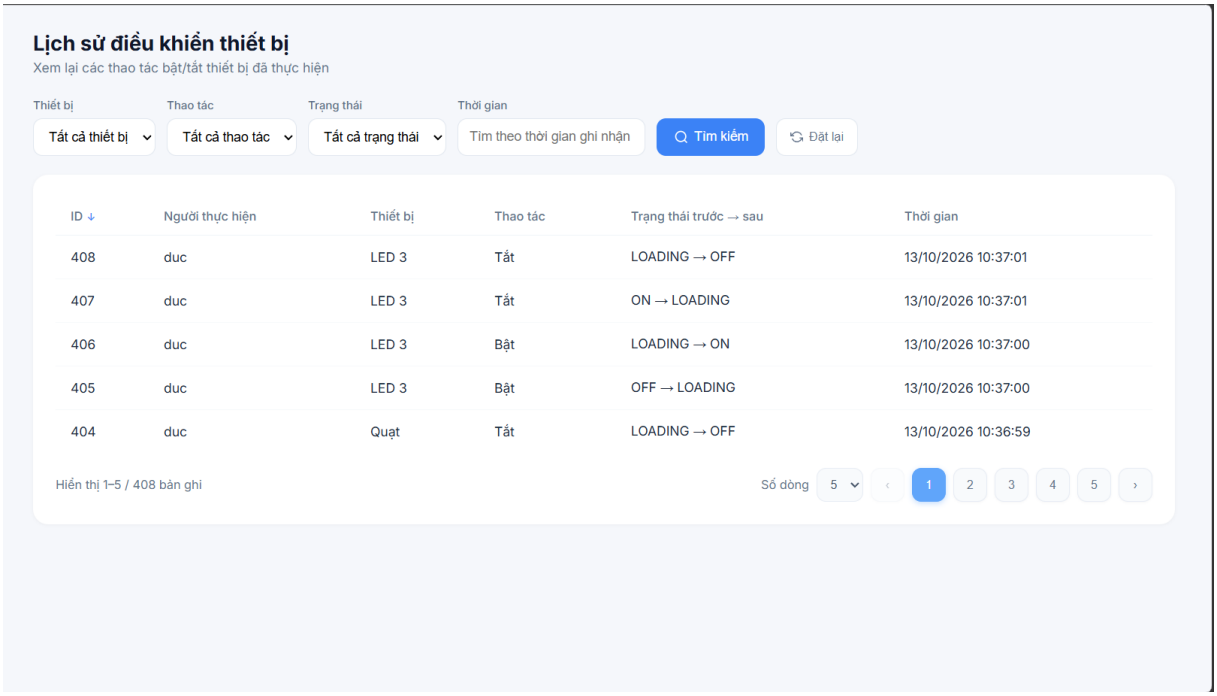
Luồng sự kiện chính

1. Người dùng truy cập chức năng xem lịch sử điều khiển thiết bị
2. Frontend hiển thị danh sách các lịch sử điều khiển
3. Người dùng cso thể chọn thiết bị hoặc nhập thời gian để tìm kiếm
4. Frontend gửi yêu cầu lấy lịch sử đến backend
5. Backend kiểm tra các tham số tìm kiếm, backend truy vấn database để lấy lịch sử điều khiển
6. Database trả về các bản ghi phù hợp
7. Backend kết hợp thông tin thiết bị và người thực hiện thao tác
8. Backend xử lý và sắp xếp dữ liệu theo thời gian
9. Backend trả kết quả về frontend, nhận dữ liệu và hiển thị lịch sử điều khiển cho người dùng

Luồng ngoại lệ

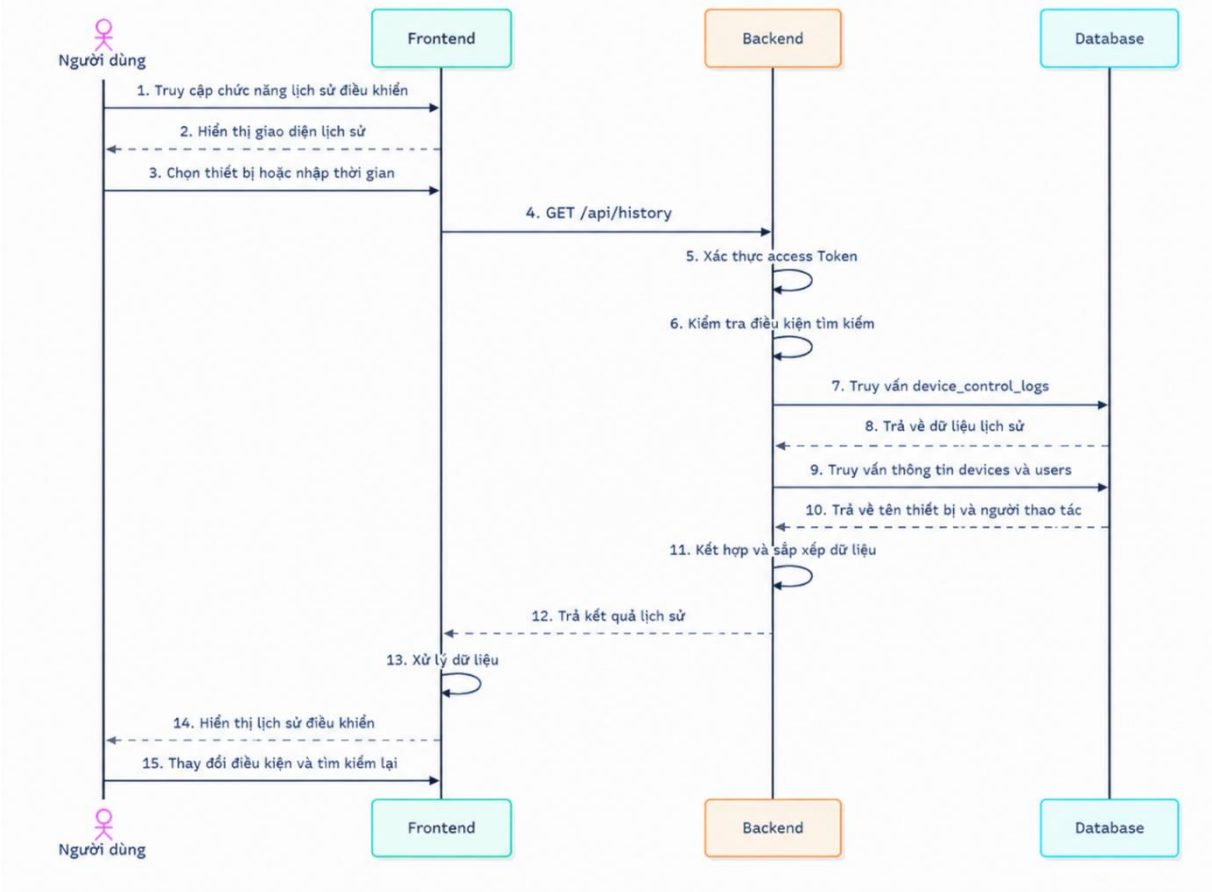
1. Thiết bị được chọn không tồn tại, backend trả về 404 Not Found
2. Không có lịch sử điều khiển phù hợp với điều khiển tìm kiếm , backend trả về danh sách rỗng
3. Không truyền điều kiện tìm kiếm hệ thống mặc định hiển thị toàn bộ lịch sử điều khiển, sắp xếp theo thời gian mới nhất

b. Các giao diện liên quan



Hình 3.9: Giao diện lịch sử điều khiển

c. Sequence Diagram



Hình 3.10 Sequence lịch sử điều khiển

Mô tả Sequence Diagram – UC5: xem lịch sử điều khiển thiết bị

1. Người dùng truy cập chức năng xem lịch sử điều khiển thiết bị
2. Frontend hiển thị giao diện lịch sử, bao gồm danh sách lịch sử và các điều kiện tìm kiếm

3. Người dùng có thể chọn thiết bị hoặc nhầm thời gian tìm kiếm. Nếu không nhập điều kiện, hệ thống sẽ lấy toàn bộ lịch sử
4. Frontend gửi yêu cầu /api/v1/devices/control-history đến backend cùng các tham số tìm kiếm nếu có
5. Backend xác thực access token của người dùng
6. Backend kiểm tra và xử lý các điều kiện tìm kiếm
7. Backend truy vấn bảng device_control_logs để lấy các bản ghi lịch sử điều khiển
8. Database trả về các bản ghi lịch sử phù hợp
9. Backend tiếp tục truy vấn bảng devices và users để lấy thông tin thiết bị và người thực hiện thao tác
10. Database trả về tên thiết bị và thông tin người thao tác
11. Backend kết hợp dữ liệu từ các bảng đồng thời sắp xếp lịch sử theo thời gian thao tác
12. Backend trả kết quả lịch sử điều khiển về frontend
13. Frontend xử lý dữ liệu nhận được
14. Frontend hiển thị lịch sử cho người dùng gồm thiết bị, người thao tác, thao tác, trạng thái trước, sau và thời gian
15. Người dùng có thể thay đổi thiết bị hoặc thời gian tìm kiếm để xem các lịch sử khác

d. Xây dựng API

Thuộc tính	Nội dung
Method + URL	GET /api/history?device=fan&action=ON&page=1&page_size=5&order=desc
Header	Authorization: Bearer <access_token>
Response	{ "data": [{ "id": 15, "device_name": "Đèn phòng", "user_name": "Nguyễn Minh Đức", "action": "ON", "previous_status": "OFF", "new_status": "ON", "created_at": "2026-10-13T10:30:15+07:00" }], "count": 13, "page": 1, "page_size": 5 }

e. Mapping API đến database

Query:

Tham số	Ví dụ	Mô tả
device	all | light | fan | device_id	Lọc theo thiết bị
action	all | Bật | Tắt | ON | OFF	Lọc thao tác
status	all | ON | OFF | LOADING	Trạng thái SAU thao tác (new_status)
time	13/10 hoặc 10:30	Chuỗi chứa trong thời gian
order	asc | desc	Sắp theo id
page	1	Số trang
page_size	5 | 8	Số dòng mỗi trang

API	Request	Bảng	Cột	Mục đích
GET /api/history		device_control_logs	id, action, previous_status, new_status, created_at	Bản ghi lịch sử
GET /api/history	device / device_id	device_control_logs	device_id	Lọc thiết bị
GET /api/history	action	device_control_logs	action	Lọc ON/OFF
GET /api/history	status	device_control_logs	new_status	Lọc trạng thái sau
GET /api/history	time	device_control_logs	created_at	Lọc theo thời gian
GET /api/history		devices	name	Tên thiết bị
GET /api/history		users	full_name	Người thực hiện

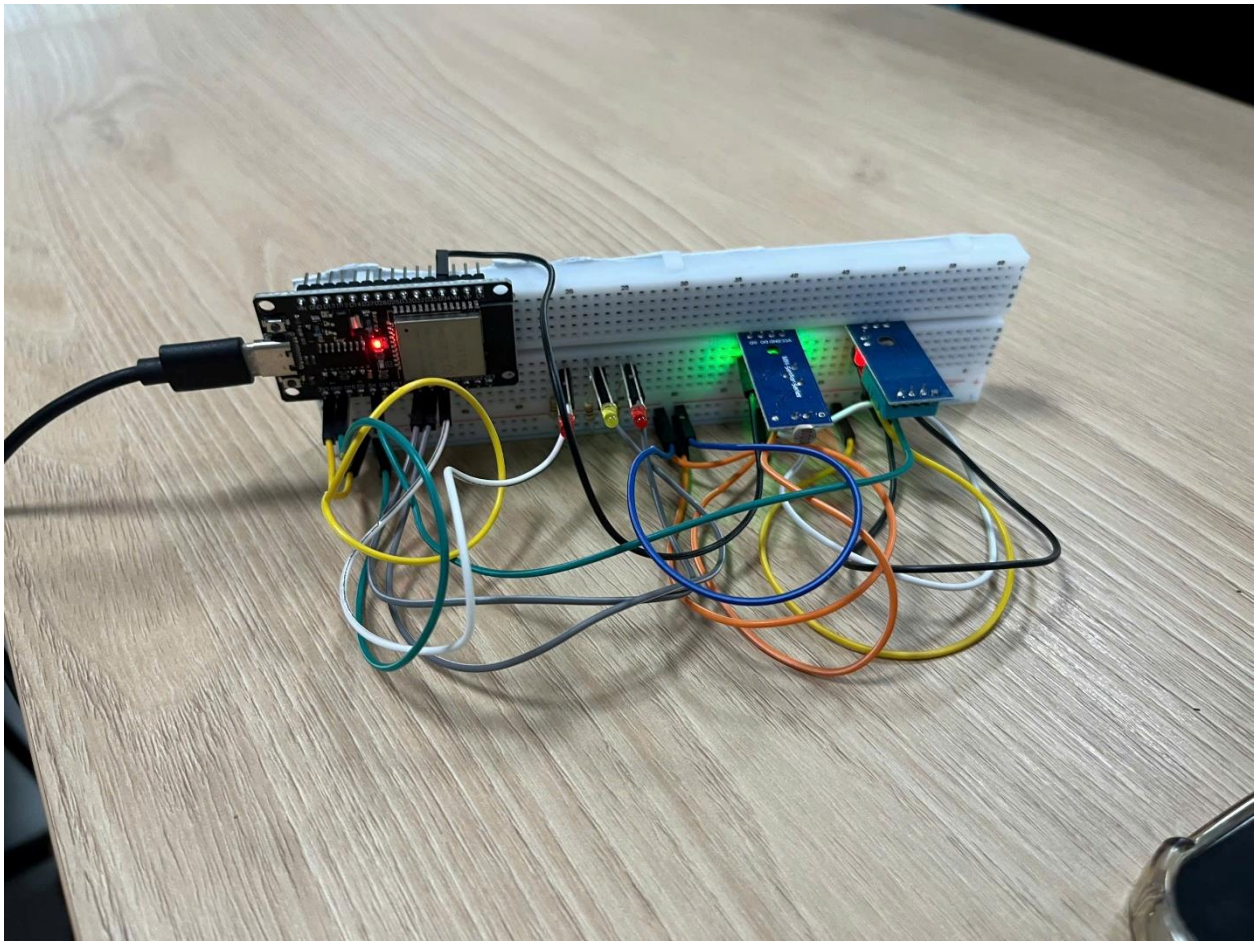
CHƯƠNG 4. KẾT QUẢ THỰC HIỆN

4.1. Kết quả triển khai hệ thống

Hệ thống **giám sát và điều khiển phòng học thông minh** đã được triển khai theo kiến trúc mô tả tại Chương 2, gồm năm nhóm thành phần: Frontend Web (HTML, CSS, JavaScript, Chart.js), Backend Django (REST API, JWT), cơ sở dữ liệu MySQL, MQTT Broker Mosquitto, và phần cứng ESP32 kèm cảm biến DHT11, quang trở (LDR) cùng các ngõ ra điều khiển đèn/quạt.

Các chức năng tương ứng năm Use Case (UC1–UC5) đã được nối thành một luồng hoàn chỉnh: đăng nhập lấy token; giám sát realtime qua API lấy bản ghi cảm biến mới nhất; tra cứu lịch sử đo; bật/tắt thiết bị qua MQTT; xem lịch sử điều khiển. Các API chính đã được kiểm tra bằng Postman và đối chiếu với giao diện Web.

a. Phần cứng và quá trình chạy thử



Hình 4.1 Kết quả lắp cảm biến

Mạch thực nghiệm sử dụng **ESP32** làm bộ điều khiển trung tâm, kết nối:

- **DHT11** (chân GPIO cấu hình trong firmware): đo nhiệt độ và độ ẩm;
- **Quang trở (LDR)** qua ADC (ví dụ chân 34): đo cường độ ánh sáng tương đối (giá trị ADC, chiều sáng tăng – tối giảm sau khi hiệu chỉnh trong hàm readLight());
- **Ba ngõ ra LED/relay** (ví dụ GPIO 18, 19, 21): tương ứng Đèn phòng (LED 1), Quạt (LED 2), LED 3.

Firmware trên ESP32 thực hiện: kết nối Wi-Fi, kết nối MQTT (host/port/user cấu hình khớp Backend), định kỳ đọc cảm biến và **publish** lên topic iot/sensor/data (cùng các topic giá trị rời nếu có), **subscribe** iot/led/# để nhận lệnh ON/OFF, và **publish** trạng thái thực tế lên iot/led/{n}/status.

Khi chạy thử, dữ liệu nhiệt độ, độ ẩm, ánh sáng xuất hiện trên Dashboard sau khi tiến trình mqtt_listener của Backend nhận message và ghi MySQL. Bật/tắt từ Web làm thay đổi GPIO trên mạch và cập nhật trạng thái trên giao diện cũng như bảng lịch sử điều khiển.

(Hình 4.1. Mô hình phần cứng sử dụng trong quá trình chạy thử — bổ sung ảnh sau.)

b. Xây dựng và tổ chức Frontend

Giao diện Web được xây dựng dạng ứng dụng một trang (SPA) với các vùng chức năng:

Giao diện	Đường dẫn (SPA)	Nội dung chính
Đăng nhập	/login	Form username/password, gọi API login
Giám sát	/dashboard	Card nhiệt độ/độ ẩm/ánh sáng, biểu đồ Chart.js, điều khiển thiết bị
Dữ liệu cảm biến	/data	Bảng lịch sử đo, lọc kiểu/giá trị/thời gian, phân trang
Lịch sử điều khiển	/history	Bảng log bật/tắt, lọc thiết bị/thao tác/trạng thái/thời gian
Profile	/profile	Thông tin sinh viên và liên kết GitHub, Figma, Postman, PDF báo cáo

Dashboard cập nhật cảm biến bằng cách **poll định kỳ** GET /api/dashboard (không dùng Socket.IO). Biểu đồ nhận thêm điểm từ cùng nguồn API. Điều khiển thiết bị gọi PUT /api/devices/{id}/control; danh sách thiết bị lấy từ GET /api/devices.

c. Luồng cảm biến và điều khiển

Chiều thu thập:

DHT11 + LDR → ESP32 → MQTT topic iot/sensor/data
→ Backend mqtt_listener (_handle_sensor)
→ MySQL (sensor_readings, sensor_data)
→ GET /api/dashboard | GET /api/data → Frontend

Chiều điều khiển:

User → Frontend → PUT /api/devices/{id}/control
→ DeviceControlView (ghi device_control_logs, cập nhật devices)
→ publish MQTT iot/led/{n}
→ ESP32 thực hiện GPIO → publish iot/led/{n}/status
→ _handle_led_status cập nhật devices.status

Trạng thái trung gian trên lịch sử điều khiển là **LOADING** (OFF→LOADING→ON hoặc ngược lại), phản ánh quá trình chờ/đồng bộ giữa Web, Backend và ESP32 — khác với mô hình PENDING/SUCCESS/FAILED của một số hệ thống khác.

d. Cơ sở dữ liệu, Backend và API

Dữ liệu được tổ chức theo năm bảng đã mô tả tại mục 2.4: **users**, **sensor_readings**, **sensor_data**, **devices**, **device_control_logs**.

Backend Django cung cấp các API chính (tiền tố /api/), xác thực JWT:

API	Phương thức	Use Case
/api/login	POST	UC1 Đăng nhập
/api/dashboard	GET	UC2 Giám sát realtime
/api/data	GET	UC3 Dữ liệu cảm biến
/api/devices	GET	UC4 Danh sách thiết bị
/api/devices/{id}/control	PUT	UC4 Bật/tắt
/api/history	GET	UC5 Lịch sử điều khiển

Tiến trình nền python manage.py mqtt_listener subscribe các topic cảm biến và status LED để ghi/cập nhật database.

e. Tóm tắt kết quả triển khai

Thành phần	Kết quả triển khai
Phần cứng	ESP32, DHT11, LDR, ngõ ra điều khiển đèn/quạt đã lắp và chạy thử
MQTT Broker	Mosquitto; topic iot/sensor/data, iot/led/{n}, iot/led/{n}/status khớp firmware
MySQL	Năm bảng theo đặc tả; có dữ liệu cảm biến và log điều khiển khi chạy thực
Backend	Django REST + JWT + mqtt_listener; các API chính phản hồi trong Postman và Web
Frontend	SPA HTML/CSS/JS, Chart.js; đủ Login, Dashboard, Data, History, Profile

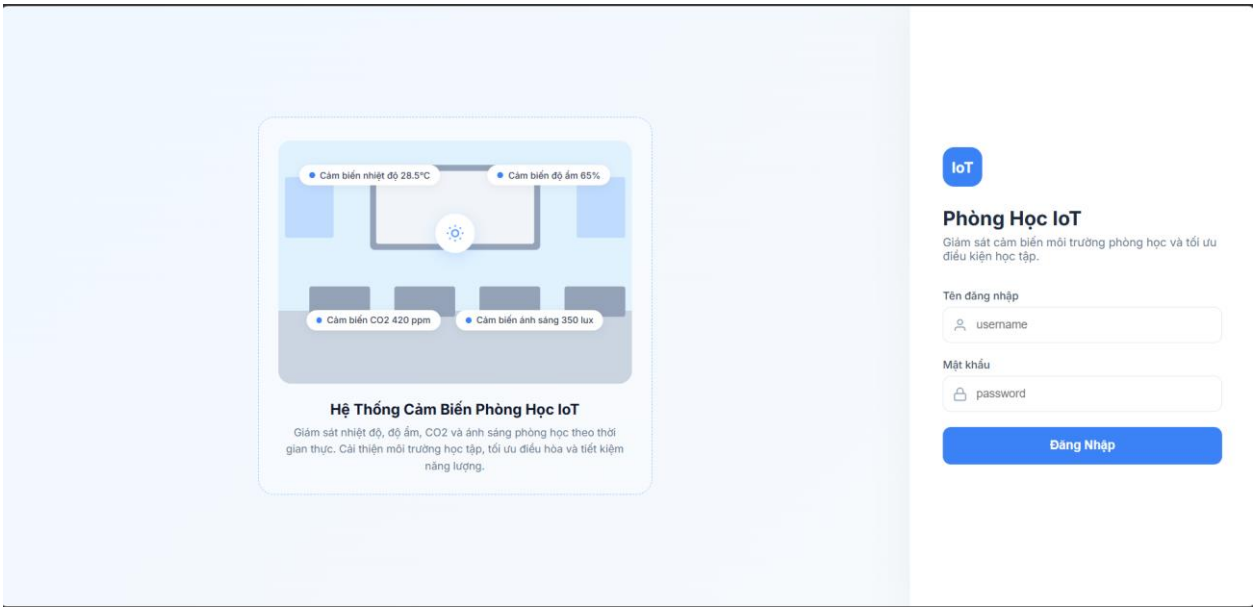
---

4.2. Kết quả giao diện và chức năng

Năm màn hình được bố trí theo đúng chức năng người dùng đã đặc tả ở Chương 3. Các hình từ 4.2 đến 4.6 minh họa giao diện sau khi triển khai; kết quả kiểm thử API trình bày tại mục 4.3.

a. Màn hình đăng nhập

Trang đăng nhập tiếp nhận tên đăng nhập và mật khẩu, hỗ trợ nhập xong nhấn Enter để gửi. Frontend gọi **POST /api/login**. Khi hợp lệ, Backend trả access token và thông tin user; Frontend lưu token và chuyển sang Dashboard. Tài khoản không đúng hoặc thiếu thông tin được báo lỗi trên giao diện.

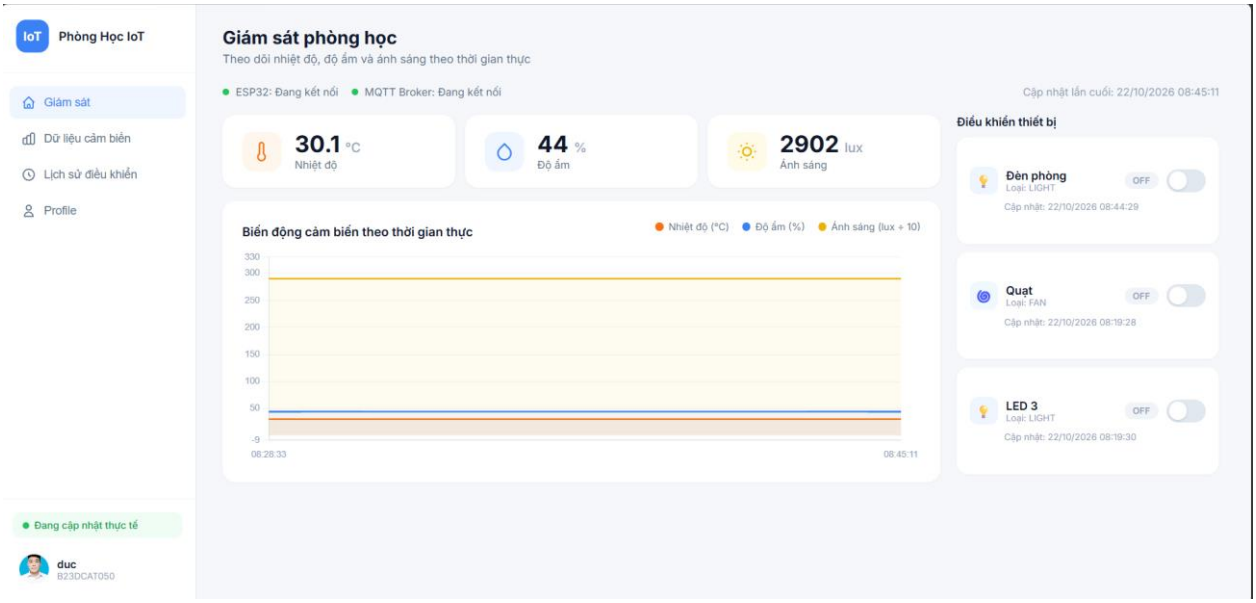


Hình 4.2 Giao diện đăng nhập

b. Dashboard – dữ liệu thời gian thực, biểu đồ và điều khiển thiết bị

Dashboard hiển thị ba card **Nhiệt độ (°C)**, **Độ ẩm (%)**, **Ánh sáng** (giá trị từ LDR/ADC, nhãn hiển thị theo giao diện), biểu đồ đường Chart.js theo thời gian, và các card điều khiển thiết bị (Đèn phòng, Quạt, LED 3, …) với trạng thái ON/OFF/LOADING và công tắc bật/tắt.

Dữ liệu cảm biến được làm mới định kỳ qua **GET /api/dashboard**. Điều khiển gọi **PUT /api/devices/{id}/control** với body {"action":"ON"} hoặc "OFF". Trạng thái thiết bị đồng bộ lại qua **GET /api/devices** và phản hồi status từ ESP32.

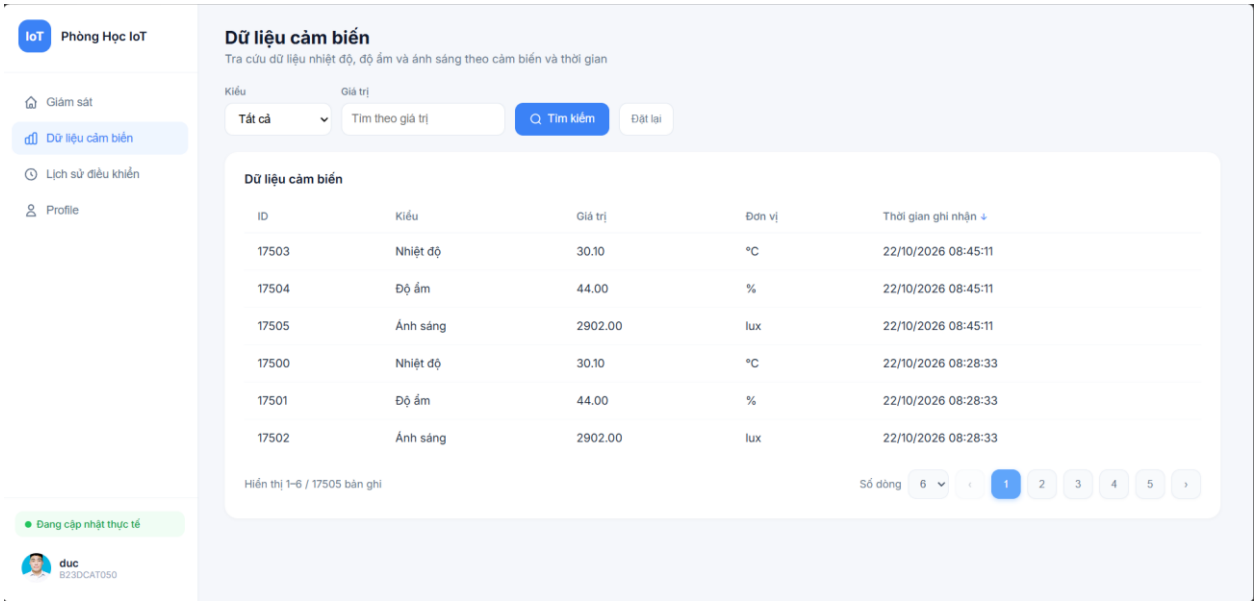


Hình 4.3 Giao diện giám sát

c. Màn hình dữ liệu cảm biến

Trang dữ liệu cảm biến hiển thị bảng các cột **ID**, **Kiểu**, **Giá trị**, **Đơn vị**, **Thời gian ghi nhận**. Người dùng lọc theo kiểu (Tất cả / Nhiệt độ / Độ ẩm / Ánh sáng / Thời gian), nhập chuỗi tìm kiếm, sắp xếp theo ID hoặc thời gian, chọn số dòng (4/6/8) và phân trang.

Dữ liệu lấy từ **GET /api/data** trên bảng **sensor_data** (mỗi dòng một loại đo: temperature, humidity, light).

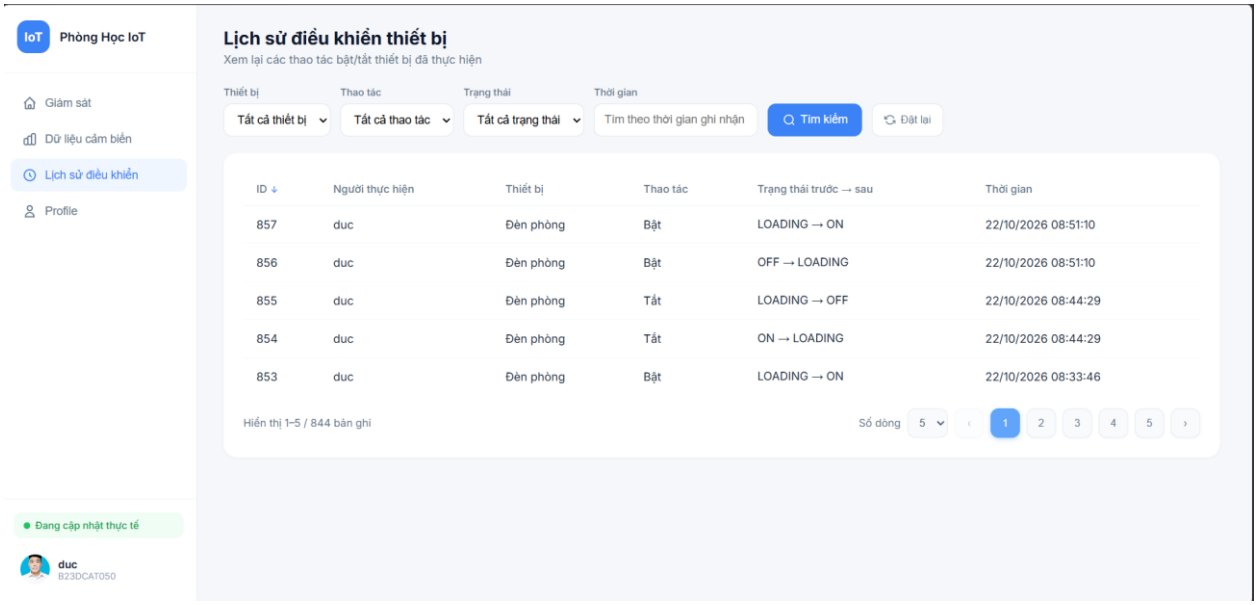


Hình 4.4 Giao diện dữ liệu cảm biến

d. Màn hình lịch sử điều khiển thiết bị

Trang lịch sử hiển thị **ID**, **Người thực hiện**, **Thiết bị**, **Thao tác**, **Trạng thái trước → sau**, **Thời gian**. Bộ lọc theo thiết bị, thao tác (Bật/Tắt), trạng thái (ON/OFF/LOADING), chuỗi thời gian; phân trang (5/8 dòng).

Dữ liệu lấy từ **GET /api/history** trên bảng **device_control_logs**, kết hợp tên thiết bị và người dùng. Log được ghi khi người dùng thao tác trên Web thông qua DeviceControlView.



Hình 4.5 Giao diện lịch sử điều khiển

e. Màn hình Profile

Profile hiển thị thông tin sinh viên thực hiện (họ tên **Nguyễn Minh Đức**, MSSV **B23DCAT050**), tên dự án Phòng Học IoT, và các liên kết: GitHub, Figma, Postman, file PDF báo cáo.

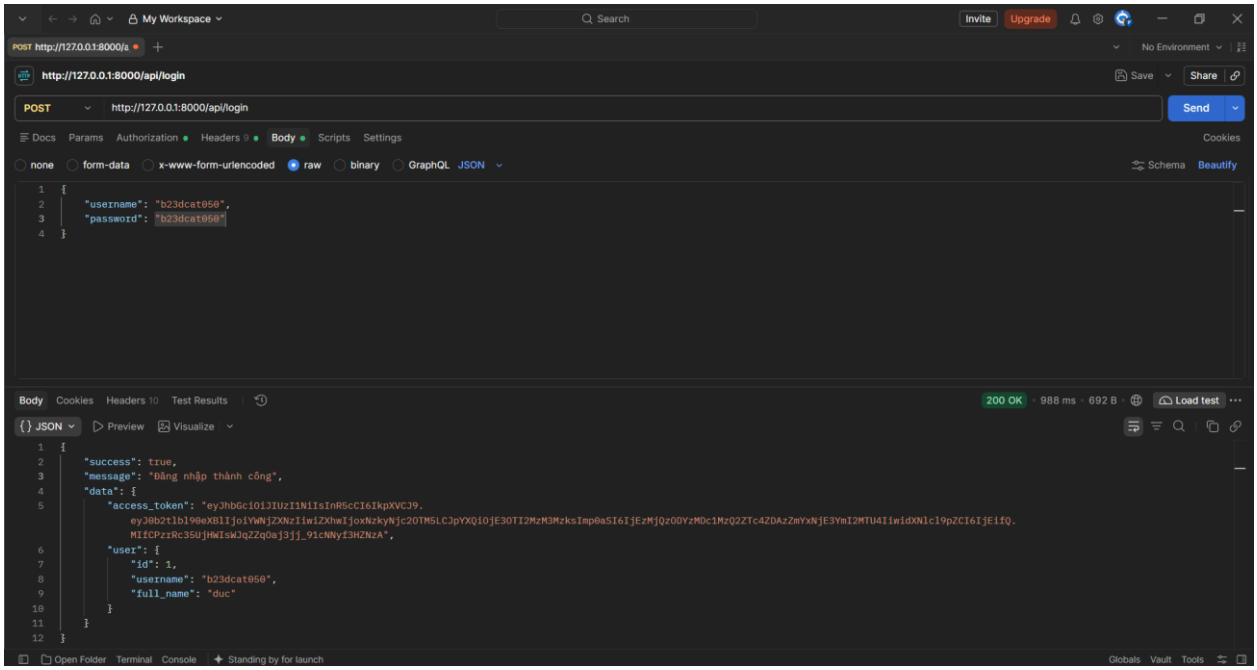
(Hình 4.6. Kết quả màn hình Profile)

4.3. Kết quả kiểm thử API

Các API chính được kiểm tra bằng **Postman** với header Authorization: Bearer <access_token> (trừ login). Dưới đây là mô tả kết quả kỳ vọng theo đúng endpoint đã triển khai; ảnh chụp Postman bổ sung vào các hình 4.7–4.13.

a. Đăng nhập — POST /api/login

- Body: {"username":"...","password":"..."}
- Kết quả thành công: **HTTP 200**, trả access token và object user (id, username, full_name, …).
- Token dùng cho mọi API tiếp theo.



Hình 4.6 Kiểm thử API đăng nhập

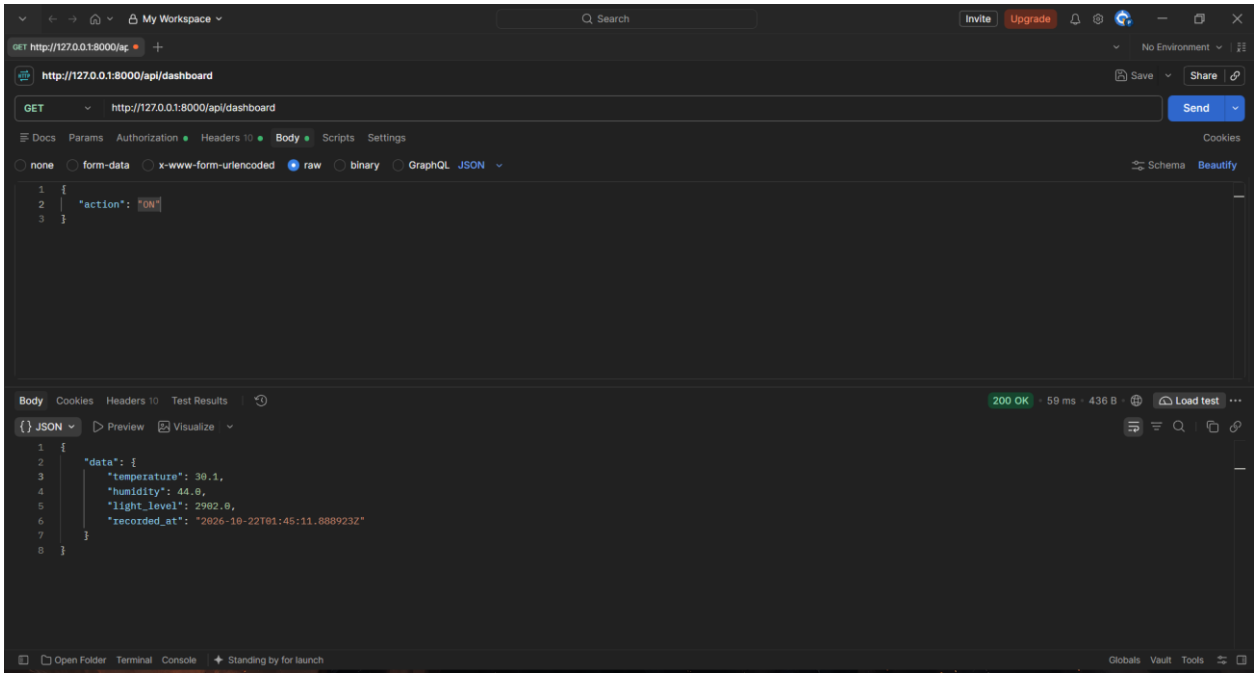
b. Dữ liệu cảm biến mới nhất — GET /api/dashboard

- Kết quả: **HTTP 200**, body dạng:

```
{
  "data": {
    "temperature": 28.5,
    "humidity": 65.0,
```

```
    "light_level": 1200,
    "recorded_at": "..."
  }
}
```

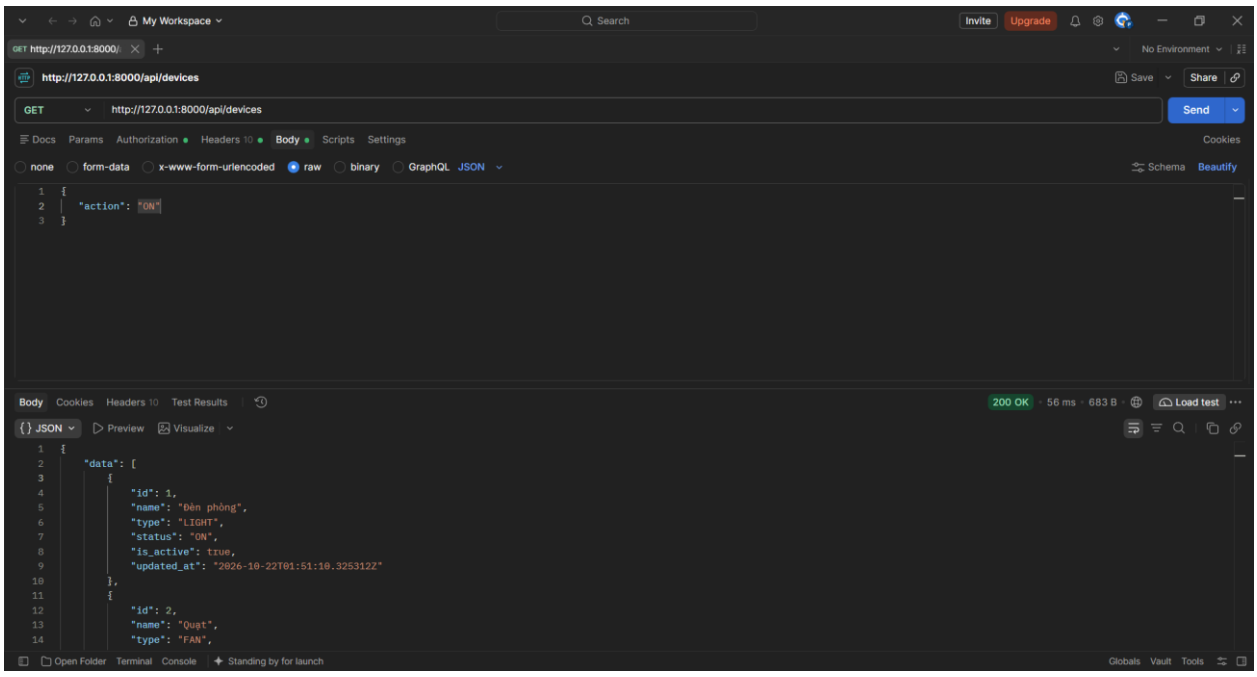
- Phục vụ card và biểu đồ Dashboard.



Hình 4.7 Kiểm thử API dashboard / cảm biến mới nhất

c. Danh sách thiết bị — GET /api/devices

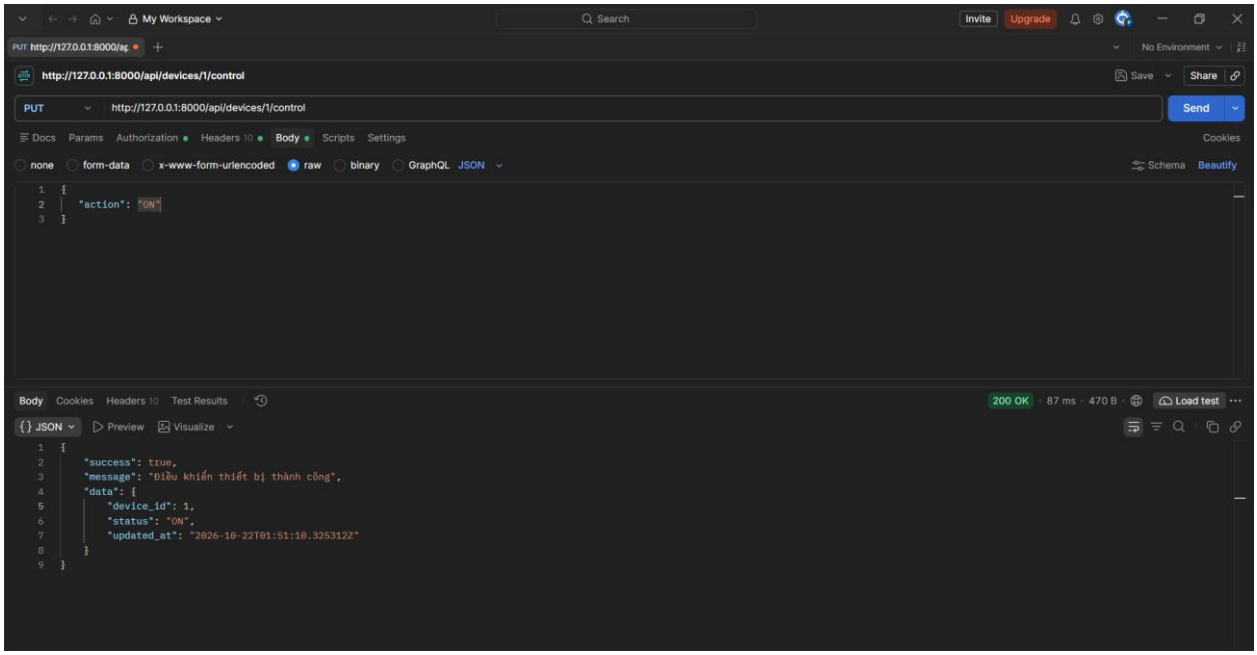
- Kết quả: **HTTP 200**, mảng thiết bị với id, name, type, status, is_active, updated_at.



Hình 4.8 Kiểm thử API danh sách thiết bị

d. Điều khiển thiết bị — PUT /api/devices/{id}/control

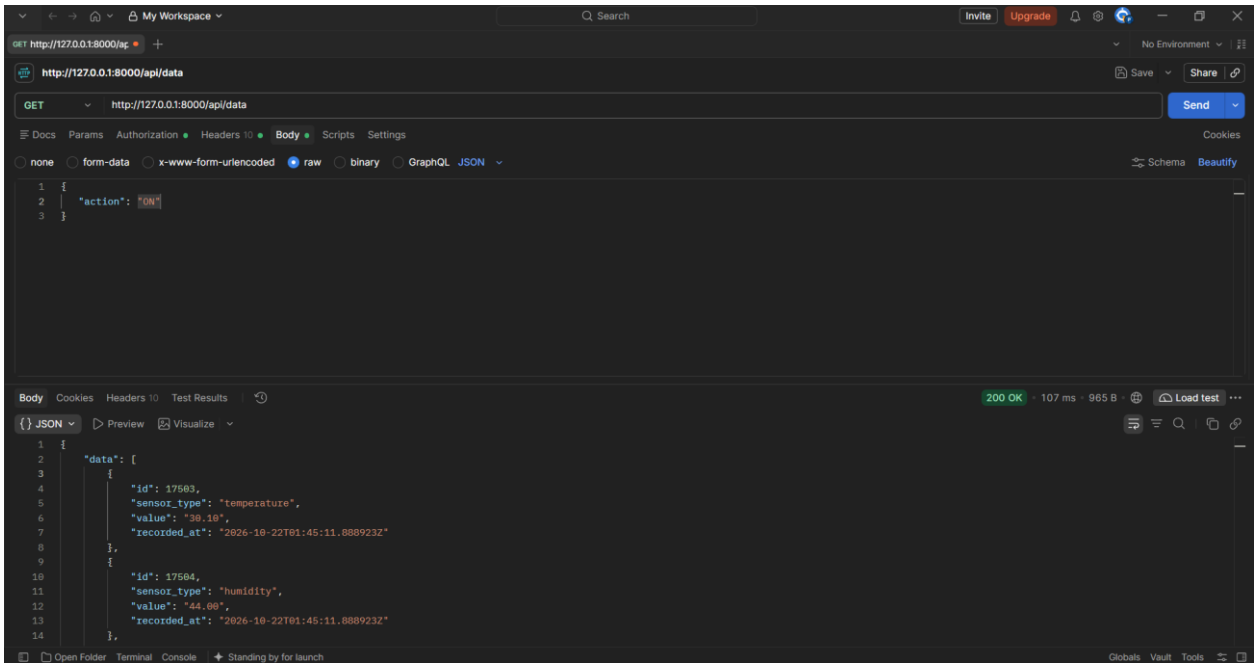
- Body: {"action":"ON"} hoặc {"action":"OFF"}
- Kết quả: **HTTP 200** (theo triển khai hiện tại), cập nhật trạng thái / ghi log; Backend publish MQTT tới ESP32.
- Lịch sử có các bước trạng thái có thể gồm **LOADING**.



Hình 4.9 Kiểm thử API điều khiển thiết bị

e. Lịch sử cảm biến — GET /api/data

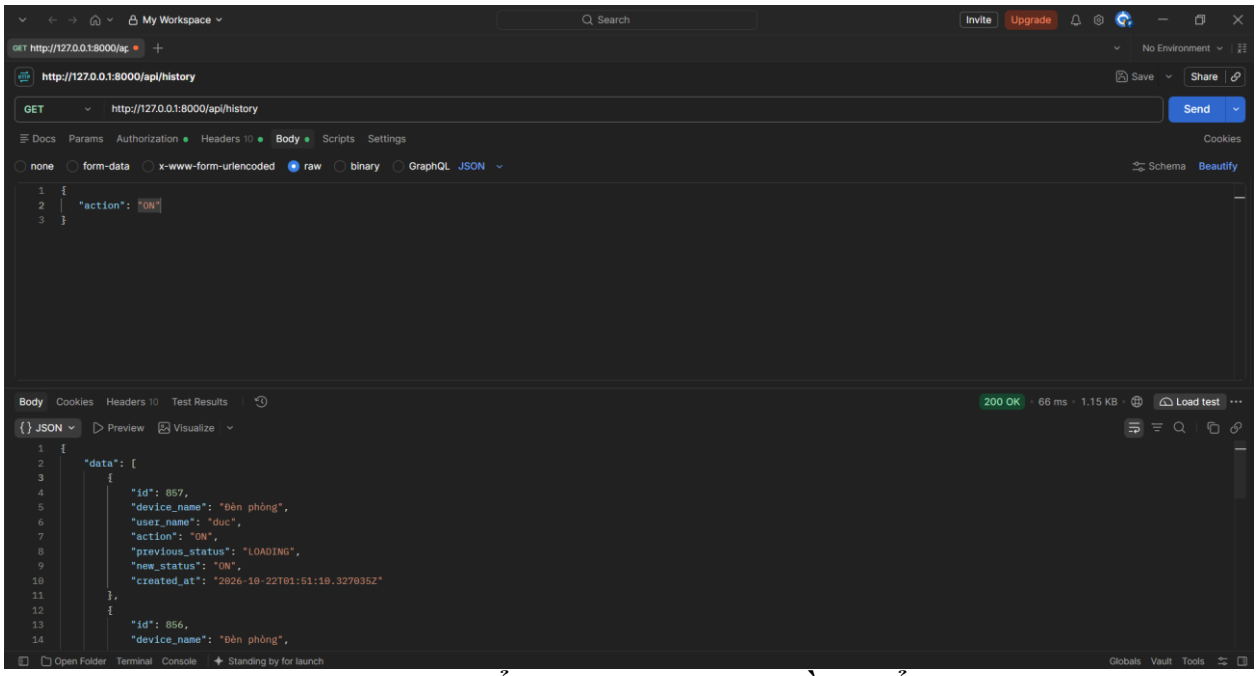
- Query ví dụ: sensor=temp&page=1&page_size=6&sort=time&order=desc
- Kết quả: **HTTP 200**, { data: [...], count, page, page_size } với từng phần tử id, sensor_type, value, recorded_at.



Hình 4.10 Kiểm thử API dữ liệu cảm biến

f. Lịch sử điều khiển — GET /api/history

- Query ví dụ: device=all&action=all&status=all&page=1&page_size=5&order=desc
- Kết quả: **HTTP 200**, các bản ghi gồm device_name, user_name, action, previous_status, new_status, created_at.



Hình 4.11. Kiểm thử API lịch sử điều khiển

g. Ghi chú kiểm thử tích hợp

- Khi ESP32 và mqtt_listener cùng online, sau vài giây Dashboard có số liệu mới; bật/tắt Web làm thay đổi đèn/quạt trên mạch.
- Khi ngắt ESP32 hoặc Broker, điều khiển có thể không phản hồi status; log vẫn ghi thao tác phía user, status thiết bị có thể kẹt LOADING cho đến khi có status MQTT trở lại.

4.4. Đánh giá kết quả

Yêu cầu	Kết quả thực hiện	Mức
Giao diện Web theo UC	Login, Giám sát, Dữ liệu cảm biến, Lịch sử điều khiển, Profile	Hoàn thành
Đăng nhập JWT	POST /api/login trả token; các API yêu cầu Bearer	Đã kiểm thử
Giám sát realtime	Poll /api/dashboard + Chart.js; nguồn ESP32 → MQTT → MySQL	Đã chạy thử
Tra cứu cảm biến	/api/data lọc, sắp xếp, phân trang trên sensor_data	Đã xây dựng
Bật/tắt thiết bị	/api/devices/{id}/control + MQTT iot/led/{n} + status	Đã chạy thử

Yêu cầu	Kết quả thực hiện	Mức
Lịch sử điều khiển	/api/history trên device_control_logs	Đã xây dựng
CSDL	Đúng năm bảng đặc tả Chương 2	Đã đối chiếu
Phần cứng	ESP32, DHT11, LDR, LED/relay	Đã lắp thử

Hệ thống đáp ứng các Use Case đã đề ra. Hạn chế: ánh sáng là giá trị ADC từ LDR (không phải lux chuẩn BH1750 trừ khi thay cảm biến và hiệu chuẩn); cập nhật realtime dựa trên polling chứ không phải WebSocket; độ ổn định phụ thuộc Wi-Fi và Broker trong môi trường thí nghiệm.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Bài tập lớn đã xây dựng được hệ thống IoT giám sát và điều khiển phòng học với các thành phần:

- **Phần cứng:** ESP32 thu thập nhiệt độ, độ ẩm, ánh sáng và điều khiển các thiết bị qua MQTT;
- **Trung gian:** Mosquitto Broker; Backend Django lắng nghe và publish topic đã thống nhất;
- **Lưu trữ:** MySQL với các bảng users, sensor_readings, sensor_data, devices, device_control_logs;
- **Ứng dụng Web:** giao diện đăng nhập, giám sát realtime (card + biểu đồ), tra cứu dữ liệu cảm biến, bật/tắt thiết bị, xem lịch sử điều khiển và trang profile;
- **API REST + JWT** phục vụ đủ năm Use Case; đã kiểm thử bằng Postman và chạy thử trên mạch thật.

Quá trình thực hiện làm rõ hai chiều dữ liệu: **thu thập** (cảm biến → ESP32 → MQTT → Backend → DB → Web) và **điều khiển** (Web → API → MQTT → ESP32 → status về Backend/DB).

5.2. Hạn chế

- Cảm biến ánh sáng dùng LDR/ADC, độ chính xác và đơn vị không tương đương cảm biến lux chuyên dụng (BH1750) nếu chưa hiệu chuẩn;
- Frontend cập nhật cảm biến theo chu kỳ poll, độ trễ phụ thuộc khoảng thời gian gọi API;
- Chưa có cơ chế cảnh báo ngưỡng, tự động hóa theo rule, hay quản lý nhiều phòng/nhiều tầng người dùng phức tạp;
- An toàn triển khai thật (HTTPS, phân quyền chi tiết, xác thực MQTT cứng) chưa phải trọng tâm phiên bản thí nghiệm.

5.3. Hướng phát triển

1. Thay hoặc hiệu chuẩn cảm biến ánh sáng sang lux; bổ sung biểu đồ thống kê dài hạn;
2. Cân nhắc WebSocket/SSE để đẩy dữ liệu realtime thay cho polling;
3. Bổ sung cảnh báo khi vượt ngưỡng nhiệt độ/độ ẩm/ánh sáng; rule tự động bật/tắt;
4. Quản lý nhiều khu vực, phân quyền admin/user rõ hơn;
5. Khi triển khai ngoài lab: HTTPS, sao lưu DB, giám sát Broker và log vận hành.

5.4. Kết luận

Đề tài đã hoàn thành mục tiêu xây dựng hệ thống giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển thiết bị phòng học qua Web, dựa trên ESP32, MQTT, Django, MySQL và giao diện HTML/CSS/JS. Các Use Case UC1–UC5 được đặc tả, thiết kế CSDL, triển khai API và kiểm thử cơ bản. Kết quả thực nghiệm cho thấy luồng dữ liệu và điều khiển hoạt động đúng kiến trúc đề ra; việc vận hành ổn định dài hạn và mở rộng tính năng là hướng tiếp theo sau bài tập lớn.

Ghi chú trình bày: Các mục "Hình 4.x" để trống chỗ chèn ảnh mạch, giao diện và Postman sau khi chụp. Có thể chỉnh số liệu mẫu (nhiệt độ, lux/ADC) cho khớp lần chạy thử thực tế của nhóm.